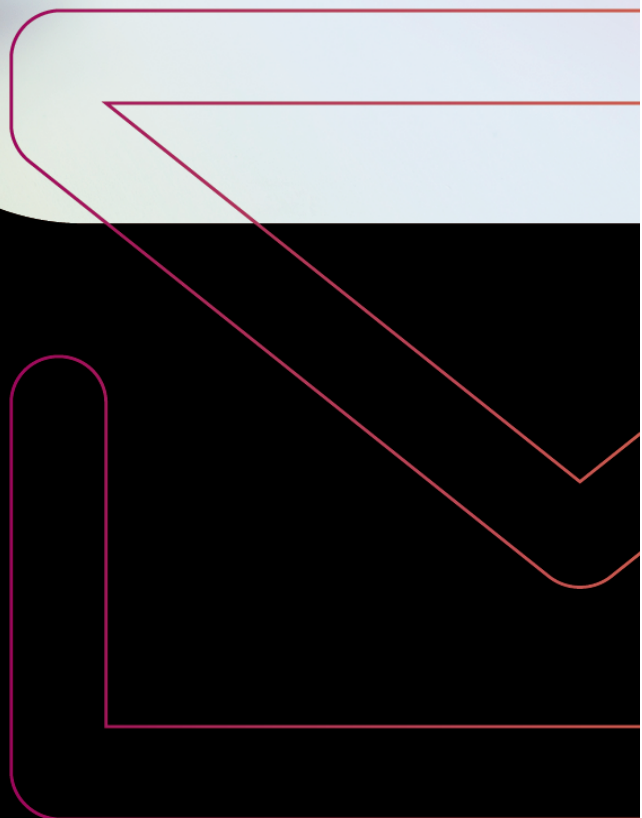


SMS API

technické rozhrania
verzia 3.1.15



eurossms



EuroSMS s.r.o.

Customer :
Contract number :
Proposal number :
Business/project number :
Project Manager : Mgr. Róbert Žilka

Issue : 3.1.15
Issue date : 1.8.2020
Status : Template
Distribution : Customer project team
EuroSMS s.r.o. project team

Prepared by	:		Róbert Žilka Date: 1.2.2018
Reviewed by	:		Martin Ďurov Date: 12.12.2018
Reviewed by	:		Jakub Mitošinka Date: 16.12.2018
Approved	:		Date:
Authorised	:		Date:
Agreed (Customer)	:		Date:

© **EuroSMS s.r.o.**

This document and its contents are the confidential property of EuroSMS s.r.o. It should not be copied, reproduced, modified, altered, or circulated to any third party, in any form or media, without the prior written consent of EuroSms s.r.o.

Amendment history

Date	Issue	Description	Author
13.10.2017	3.00	Introduction, formating template	Róbert Žilka
20.04.2018	3.01	Chapter Email	Róbert Žilka
29.10.2018	3.02	Chapter JSON – API v. 3.x	Róbert Žilka
20.11.2018	3.03	Chapter WebServices – API v. 2.x	Róbert Žilka
01.12.2018	3.04	Appendices – GSM, Send Error, Status Error	Róbert Žilka
05.12.2018	3.05	Finalisation WebService chapter	Róbert Žilka
12.12.2018	3.06	HTPPs chapter, IM chapter	Róbert Žilka
13.12.2018	3.07	CallBack chapter	Róbert Žilka
21.12.2018	3.1.10	Document finalisation	Róbert Žilka
08.01.2019	3.1.11	Úprava, doplnenie kapitoly 8.7	Róbert Žilka
14.01.2019	3.1.13	Doplnenie príklad JSON doručanky 6.1.5	Róbert Žilka
01.08.2020	3.1.15	Opravy v kapitlove WebServices / SendOne	Róbert Žilka

The table of contents

1	Manažérske zhrnutie.....	10
2	Úvod	11
2.1	Ciele a rozsah.....	11
2.2	Readership.....	11
2.3	Odkazy na iné dokumenty.....	11
3	SMS rozhrania.....	13
3.1	Prológ	13
3.2	Princíp Short Message Service a Segmentácia.....	13
3.3	Kódovanie.....	14
3.4	Technológia.....	14
4	Digitálny podpis.....	15
4.1	Úvodné pojmy.....	15
4.2	Výpočet digitálneho podpisu.....	15
4.2.1	Postup výpočtu digitálneho podpisu pre <i>jedného</i> príjemcu.....	16
5	Rozhrania Email → SMS.....	17
5.1	Typy email rozhraní.....	17
5.1.1	Metóda InTo.....	18
5.1.2	Metóda InSubject bez digitálneho podpisu.....	19
5.1.3	Metóda InSubject s digitálnym podpisom.....	19
5.1.4	Metóda ReceiveByMail.....	20
5.1.5	Príklad príjmovej sms mailom.....	20
6	CallBack (Push) notifikácie.....	21
6.1	Úvod k doručenkám.....	21
6.1.1	Metóda Doručeniek formou CallBack (Push notifikácia).....	21
6.1.2	Formát CallBack plain text.....	22
6.1.3	Formát CallBack JSON.....	22
6.1.4	Parametre.....	22
6.1.5	Príklad JSON doručeny.....	23
6.1.6	Očakávaná odpoveď zákazníckeho systému.....	23
6.2	Úvod k prijatým správam.....	23
6.2.1	Formát CallBack plain text.....	24
6.2.2	Formát CallBack JSON.....	24
6.2.3	Posielané parametre CallBack.....	24
6.2.4	Očakávaná odpoveď zákazníckeho systému.....	24
7	Rozhrania WebServices (ver. 2.x).....	25
7.1	Úvod k Webservice (WS).....	25
7.2	Typy Webservice.....	26
7.3	Posielanie jednej SMS volaním parametrami Operácia smsSendOne.....	27
7.3.1	Parametre volania smsSendOne.....	27
7.3.2	Príklad SOAP Request.....	28
7.3.3	Odpoveď systému operácie smsSendOne.....	28
7.3.4	Príklad odpovede systému operácie smsSendOne.....	29
7.3.5	Príklad SOAP Response.....	29
7.4	Posielanie jednej SMS vo formáte XML Operácia smsSendOneXML.....	30
7.4.1	XML formát.....	30

7.4.2	Atribúty.....	30
7.4.3	Elementy.....	30
7.4.4	Príklad <i>XML Request</i>	31
7.4.5	Odpoveď systému operácie smsSendOneXML.....	31
7.4.6	XML elementy odpovede.....	31
7.4.7	Príklad <i>XML Response</i>	32
7.5	Spracovanie doručeníek Operácia smsStatus.....	32
7.5.1	Parametre operácie smsStatus.....	32
7.5.2	Návratové hodnoty.....	32
7.5.3	Príklad návratového poľa.....	33
7.6	Hromadné spracovanie doručeníek XML Operácia smsStatusXML.....	33
7.6.1	Príklad <i>XML Request</i>	33
7.6.2	Odpoveď <i>Response XML</i>	34
7.6.3	Príklad <i>XML Response</i>	34
7.7	Príjem jednej správy Operácia smsReceiveOne.....	35
7.7.1	Dvojcestné správy.....	35
7.7.2	Volanie operácie smsReceiveOne.....	36
7.7.3	Návratové hodnoty poľa prijatých správ.....	36
7.7.4	Príklad prijatej krátkej správy.....	37
7.7.5	Príklad prijatého segmentu dlhej správy.....	37
7.8	Hromadný príjem správ XML Operácia smsReceiveMoreXML.....	37
7.8.1	Parametre volania smsReceiveMoreXML.....	37
7.8.2	Prototyp (príklad) tvaru generovanej funkcie smsReceiveMoreXML.....	38
7.8.3	Odpoveď operácie smsReceiveMoreXML.....	38
7.8.4	XML elementy root objektu.....	39
7.8.5	XML element rcvd.....	39
7.8.6	XML element segmentation.....	39
8	Rozhrania HTTPs (ver. 2.x).....	41
8.1	Parametre posielané v <i>Requeste</i>	41
8.2	Poslanie SMS zoznamom parametrov.....	42
8.2.1	Odpoveď systému.....	43
8.2.2	Príklady odpovedí typu <i>Plain</i>	43
8.3	Správy posielané vo formáte XML.....	43
8.3.1	Odpoveď systému na správy posielané vo formáte XML.....	44
8.3.2	Príklady odpovedí verzie <i>XML</i>	44
8.4	Vyžiadanie doručienky typu <i>Plain</i>	44
8.4.1	Odpoveď na vyžiadajú doručienku typu <i>Plain</i>	45
8.4.2	Príklad doručienky typu <i>Plain</i>	45
8.5	Vyžiadanie doručienky(niek) typu <i>XML</i>	45
8.5.1	Formát <i>XML</i> pre vyžiadanie doručeníek.....	46
8.5.2	Príklad <i>XML</i> pre vyžiadanie doručeníek.....	46
8.5.3	Odpoveď <i>XML</i> pre vyžiadanie doručeníek.....	46
8.6	Prijatá správa - typ Requestu <i>Plain</i>	46
8.6.1	Príklad volania.....	46
8.6.2	Odpoveď systému.....	47
8.6.3	Príklad odpovede prijatej správy vo formáte <i>Plain</i>	47
8.7	Prijaté správy - typ Requestu <i>XML</i>	47
8.7.1	Formát <i>XML</i> requestu pre prijaté správy.....	47
8.7.2	Príklad <i>XML</i> objektu pre príjem.....	48
8.7.3	Odpoveď systému <i>XML</i>	48

9	Rozhrania RestFul (ver. 3.x).....	49
9.1	Princípy.....	49
9.1.1	Web Application Description Language (WADL).....	49
9.2	JSON 50	
9.2.1	Request JSON root objekt.....	51
9.2.2	JSON objekt typu <i>msg</i>	53
9.3	RestFul services – posielanie správ.....	55
9.3.1	Request JSON objekt one.....	56
9.3.2	Response JSON objekt one.....	58
9.3.3	Request JSON objekt o2m.....	59
9.3.4	Response JSON objekt o2m.....	60
9.3.5	FULL <i>JSON Response</i> – kompletne zamietnutá.....	60
9.3.6	FULL <i>JSON Response</i> – zamietnuté niektoré tel. čísla.....	61
9.3.7	FULL <i>JSON Response</i> – kompletne akceptovaná.....	61
9.3.8	Request JSON objekt m2m.....	62
9.3.9	Response JSON objekt m2m.....	63
9.3.10	Tvar BASIC <i>JSON Response</i>	63
9.3.11	Tvar FULL <i>JSON Response</i>	64
9.4	RestFul services – stav doručenia.....	64
9.4.1	Status jednej správy (operácia one).....	64
9.4.2	Tvar <i>status/one Response</i>	65
9.4.3	Hromadný status správ (operácia any).....	65
9.4.4	Status skupinových/hromadných správ (operácia group).....	67
9.4.5	Tvar <i>JSON Response</i> pre operácie <i>status/group</i> a <i>status/any</i>	68
10	Instant messaging (IM).....	69
10.1	Princípy.....	69
10.2	Podmienky posielania IM.....	70
10.2.1	Príklad <i>JSON Request</i> pre posielanie IM.....	71
10.3	JSON objekt typu im.....	72
10.4	Default (predvolené) hodnoty IM – rozšírenie root objektu.....	73
10.5	Príznačky (flags) špecifické pre IM.....	74
10.6	Rozšírenie chybovníka posielania SMS o špecifické IM statusy.....	75
10.7	<i>JSON Response</i> – odpoveď systému na IM <i>Request</i>	75
10.8	Stavy doručenia IM – rozšírenie stavov doručenia.....	75
10.9	Správy prijaté Viberom.....	76
11	Apendix A.....	77
11.1	Príklady výpočtu digitálneho podpisu SHA1 (Java).....	77
12	Apendix B.....	78
12.1	Základná znaková sada GSM 3.38.....	78
12.2	Rozšírenie znakovkej sady GSM 3.38.....	79
13	Apendix C.....	80
13.1	Zoznam stavov poslania/príjmu SMS a IM.....	80
13.2	Kompletný zoznam stavov doručenia SMS, IM.....	81
14	Apendix D.....	82
14.1	Flags – príznaky ovplyvňujúce chovanie SMS API.....	82
14.2	Flags - príklady výpočtu.....	83

1 Management summary

The aim of the document is to describe the SMS interface. The operators use the SMS interface to connect to the external systems and to send short, automated messages from the customer systems. The interface communicates with the webservices in the XML format using the https, Restful, WebService, and mail protocol.

2 Introduction

2.1 Objectives and scope

The aim of this document is to describe the interface intended for sending and receiving text messages (SMS) via the Internet to the mobile phones. Additionally, a digital signature algorithm that is used for verifying the authenticity of the sending entity is described.

2.2 Readership

The document is intended for technical use only. More specifically, the document is intended for the implementation of a customer solution focusing on the connection of the external systems to the SMS gateway via an automated interface.

2.3 Links and references

For more detail regarding the terms and conditions of the SMS Gateway use, please see: General Terms and Conditions:
<http://www.eurosms.com/docs/vop.htm>

3 SMS interfaces

3.1 Prologue

To send and receive SMS, API interface uses the following methods to connect to the customer services:

- SMPP – connection provided via native SMPP protocol,
- the use of transactional email system → SMS – sending and receiving SMS via email (5 Email interfaces → SMS),
- SMS API – programming interface based on:
 - WebServices (7 WebServices Interfaces (2.x version)),
 - plain HTTPs GET/POST - partially compatible with SMS API 1.x (8 HTTPs interfaces (ver. 2.x)),
 - RestFull services (JSON) (9 RestFul (ver. 3.x) interfaces).

3.2 The fundamentals of Short Message Services and Segmentation

SMS (Short Message Service) are short messages that are deliverable to any device not just a mobile phone (e.g., telemetry and remote-control systems). SMS has not only the ability to receive messages but also to acknowledge receipt of a message to the operator.

A length of a single text message is 140 bytes, which is referred to as a segment. This practically means that a single message:

- can contain up to 160 characters from latin alphabet excluding diacritical characters (which includes special characters such as řĺšćťž...),
- can contain up to 70 characters including diacritical characters. This is mostly used in the following cases: a.) essential to maintain the original content of the message, b.) the message contains special characters (such as ÅĂĞØ...), c.) the message uses the Russian or Japanese alphabet or other characters such as Roman characters.

Due to the SMS specification (protocol SMPP version 3.4), messages that are longer than 1.) 160 characters and do not contain any diacritical characters, or 2.) 70 characters and do diacritical characters will be split into segments in the following way:

- the text of the message has 153 characters, while using non-diacritical characters
- for messages that contain at least one diacritical or special character the length corresponds to 67 characters.

The remaining 7 or 3 characters respectively, are used to re-assemble the segments into one final SMS which will be sent to the mobile phone. This final SMS will be then displayed on the beneficiary's device.

It is not recommended to send messages that are split into more than 4 segments. Instead, it is recommended to send messages with the maximum length of 612 or 268 characters, respectively. This is because messages that are split into more than 4 segments might cause message memory overflow. As a result, *such messages will not be displayed* on the beneficiary's phone at all or they will be delivered numerous times.

At the same time, **a single segment** corresponds to **one billing unit**.

3.3 Coding

The term coding in the context of SMS technology refers to the message format that is forwarded to the operator for delivery and received by the customer connections to the SMPP protocols.

Generally, all interfaces use UTF-8 coding (mail, Restful, WebServices, HTTP). However, GSM 3.38 is default for SMPP.

Messages from the EuroSMS technology are sent to the operators/ aggregates by using the following:

- GSM 3.38, unless diacritical characters are required
- UCS2 (UTF) for full support of diacritical and special characters such as Cyrillic script, Russian alphabet and so forth and so on.

For more details regarding the GSM 3.38 character set i.e., all SMS without diacritical characters, please see 12 Appendix B.

3.4 Technology

EuroSMS s.r.o. uses strictly its own hardware and inhouse developed software.

All systems are service oriented. Enterprise Java Applications use asynchronous technologies, such as Java messaging etc. Almost all components are redundant including software and hardware.

Overall, the hardware is grouped into the clusters, that are currently located in the best housing centres such as Digitalis and Telekom data centres. These centres are secured by two-circuit independent power supplies as well as multiple internet connections. The switches, routers, network cards, all disks in RAID 1, 5, and 6 disk arrays are all redundant.

While servers and Blade Servers are exclusively IBM/Lenovo brands, the switches and routers are strictly Cisco brands.

4 Digital signature

4.1 Introductory terms

The digital signature consists of characters created from the beneficiary's phone number and the private key that belongs to the customer identifier.

For the purpose of integration, EuroSMS uses the electronic channels (sms, email) to create and deliver the following to the customers:

- **Integration identifier** is also known as Integration ID. Normalized format, CASE SENSITIVE, format: X-YYYYYYY,
- where X can take the digital values of [0..9],
- YYYYYY are ASCII characters {[0..9] [a..z] [A..Z] [-]}
- **Integration key** - contains at least 8 characters from the set of {[0..9] [a..z] [A..Z] [-|()/^] }, CASE SENSITIVE.

The integration ID and the integration Key have their distinctive formats. It is important to note, that the integration key and ID are not in any way related to the login name and password to the portal, and therefore they cannot be used for SMS sending via automated interfaces.

Each customer account and sub-account is assigned a unique *Integration ID* and *Key*.

4.2 Digital signature calculation

The digital signature is used to verify the authenticity of the customer system request to send a SMS message(s). The digital signature is never identical to login name or password to the <https://www.eurosms.com> portal.

An algorithm based on HMAC_SHA1 or HMAC_SHA256 is used to create the digital signature. The system automatically detects the used algorithm (briefly, for example, at <https://sk.wikipedia.org/wiki/HMAC>).

The digital signature is calculated by using the **integration key**, **beneficiary's telephone number**, **sender's telephone number**, and **SMS text**.

4.2.1 The digital signature calculation procedure used for a single recipient

The attributes used to calculate the digital signature:

Attributes' title	Value	Sent in request
Sender	RZi	✓
Addressee	421903622237	✓
Text of the message	Testovacia sprava	✓
Integration identifier	2-A2gHjk	✓
Integration key	Gh-s7-J6	✗

Digital signature calculation procedure:

1. Sender, recipients' phone number, and the messages are joined into one continuous string: **Rzi421903622237Testing message**
2. the following key **Gh-s7-J6** will be applied to the above string using the **HMAC_SHA1** algorithm **6f56060b6b7db97ca25782b771cca0a65077bd5b**
3. the string obtained in the step 2 is the digital signature corresponding to the text-message ~~that will be send to the recipient~~

The calculation of the *digital signature* for the bulk and group messages (9.3.3 Request JSON object o2m) is provided in the respective parts of the document

The final *digital signature* is **not** CASE SENSITIVE.

The server simultaneously performs an calculation identical to the clients' calculation. The customer service **never** sends the integration key. Instead, the service sends the *Integration ID*. Based on the *Integration ID* the server searches for the *Integration Key*. Subsequently, the digital signature is calculated using the *Integration Key*.

If the digital signature generated by the server is identical to the digital signature sent by client (regardless of the character size), the request is considered correct and will be further processed.

5 Email interfaces → SMS

5.1 Types of email interfaces

Email-type interfaces→ SMS provide services to send and receive SMS via email.

There are two methods that can be used for sending SMS via email:

- The beneficiary's telephone number is part of the email address (5.1.1 The InTo method), which can be used with or without the digital signature. The method allows sender to send a single message by email.
- The list of beneficiary's phone numbers is listed in the email subject. This method allows the sender to send multiple messages to several phone numbers.
- Two variants are available:
 - 5.1.2 InSubject Method without digital signature ,
 - 5.1.3 InSubject Method with digital signature .

The operators' virtual numbers are used to receive SMS via email, from which the systems receive messages and deliver them to the email addresses owned by the client (5.1.4 ReceiveByMail method).

5.1.1 The InTo method

To send a SMS one has to insert a phone number into the recipients' (to) message, where the domain is email.to.mobi. Emails are sent to an email address such as 0903622237@email.to.mobi. The system is able to send messages of any length, however, the text must be recognizable by the system and located in the body (message content/ body) of the message. For the billing purposes, it is essential that the message does not exceed the maximum permitted length of the message (3.2 The fundamentals of Short Message Services and Segmentation) The system searches for the code (in email Subject field) or for the digital signature listed in the subject of the message.

The email arrays:

- **From** (sender) – irrelevant but mandatory value. This array is used to report the method that was used to process the message. The system does not respond to addresses such as noreply@xxxx.yy.
- **To** (receiver, addressee) – contains the correct format of recipients' telephone number. The recommended format of the phone number is a full phone number including a prefix. The correct formats are shown below:
 - +421903622237@email.to.mobi,
 - 00421903622237@email.to.mobi,
 - 0903622237@email.to.mobi¹,
 - 903622237@email.to.mobi.

The phone number formats are reviewed directly at the input.

Incorrect numbers (incorrect format, abbreviated numbers, purchased phone numbers, geographical numbers etc.) are rejected directly at the input. Thus, SMS sent to an incorrect phone numbers will not be charged.

- **Subject** (predmet) – contains either:
 - special code that is assigned on request,
 - combination of X-ZZZZZZ:yyyyyyyyyyyy, where X-ZZZZZZ is assigned an integration key and yyyyyyyyyyy is a digital signature (see Digital signature calculation)
- **Body** (content) – contains the text which is sent in the SMS format to the phone number. The phone number is stated in the 'To' array. Formats:
 - **Plain** (plain text) – recommended text format,
 - **HTML** – send by many mail systems. In this case, the system will attempt to select the part that contains only text.
 - **Multipart** – this includes messages that e.g., not only contain text but also annexes. In this case, the non-text content is ignored.

1 Skrátené formáty (bez prefixu krajiny) sú použiteľné výhradne pre Slovensko a Česko.

5.1.2 InSubject Method without digital signature

In order to send a SMS it is essential to insert list of telephone number/s in its correct format, and separate the digits by permitted numeric separators into the message subject. Subsequently, the message should be sent to the following email address: email2sms@eurosms.com.

The system verifies the server and email address of the sender in order to ensure minimal security. The email is further processed if the above conditions are met. In other words, the systems considers the requester to be authentic if an existing server and sender is detected and have enabled

The system verifies the server and email address of the sender, in order to ensure minimal security. The email is further processed if the above conditions are met. In other words, the system considers the requester to be authentic if an existing server and sender is detected and have enabled the InSubject Method without digital signature . Please send your request to set up this method to support@eurosms.com.

Warning: From the security point of view, emails of this type are not comprehensively verifiable and therefore this method is generally not recommended.

The system can send message of any length read from the Body (content/body of the message). For the billing and delivery purposes, it is essential that the length of the message does not exceed the maximum permitted message length (3.2 The fundamentals of Short Message Services and Segmentation). Does not allow to insert the senders' signatures, and/or results of antivirus scan results, etc. into the customer systems. The system automatically removes some of the frequently used antivirus appendices from the text.

The system is able to send message of any length that is read from the Body (content/body of the message). For the billing and delivery purposes, it is essential that the length of the message does not exceed the maximum permitted message length

The system looks up the sender of the SMS based on the senders' address.

The following symbols are numeric separators that are permitted by the system: | , ; :

The numeric separators can be combine within one request. Two consecutive numerical separators (without any other character in between) are viewed as one. The numbers can be presented in any format (vid'. 5.1.1 The InTo method, part **To**).

5.1.3 InSubject Method with digital signature

Where IntegrationID in the format of 1-XXXXXX or 2-YYYYYY is assigned to EuroSMS and the digital signature is calculated by adding IntegrationId, Subject, and Body.

The IntegrationId, Subject, and Body are used in the original format in which they were sent by email.

The phone number/numbers are saved in the Subject of the message as stated in 5.1.2 InSubject Method without digital signature .

5.1.4 ReceiveByMail method

Upon request, *ReceiveByMail* method allows to send messages delivered to the virtual numbers to the defined email (company to which a virtual number was assigned).

Received SMS arrives by email in its completed format regardless of the initial message length. Thus, even long messages will be delivered to the email in their complete format.

The email contains:

- sender (telephone number),
- delivery time,
- the text of received message.

It is possible to send messages to multiple emails simultaneously, upon request.

A receipt emails is configured and tested by EuroSMS in accordance with the customers' request. The customers' requests can be sent to the following email address: support@eurosms.com.

***Warning:** EuroSMS is unable to grant or control the email delivery to the customers' mailbox.*

5.1.5 An example of the incoming SMS by email

```
Odosielať: 421903622237
Prijemca: 421902022088
Čas prijatia: 2017-07-18 15:46:35

Správa:
-----
Lekarka odporúča:
- nádychov 5x ráno/večer (120 dávok)
- rhynocord ráno/večer
- alerphid 1x denne (ráno/večer)
-----
Mail bol generovaný automaticky, neodpovedajte naňho.
```

6 Callback (Push) notifications

Callback also known as Push notifications are used to deliver information to the customer systems without the customers' explicit request. Thus, a change within EuroSMS systems, will result in an automated Callback notifications providing customers with available information regarding current situation.

For the purpose of Callback notifications, http(s) calls or mail notifications are used:

- http(s) Callback delivery reports
6.1.1 Callback delivery report method (Push notification),
- http(s) Callback for received messages
6.2 Introduction to received messages
- mail notifications for received messages
5.1.4 ReceiveByMail method

6.1 Introduction to the delivery reports

A delivery report is a complex information obtained from the operator. This information contains information regarding the message status (i.e., was the message delivered, is the phone number correct,...) as well as the price of the message/segment or the identity of the operator.

The delivery reports are promoted to the customer systems by using the following methods:

- requesting HTTPs method (8.4 Requesting Plain delivery report alebo 8.5 XML delivery report(s) request),
- requesting Webservice operations (7.5 Delivery report processing SmsStatus operation or bulk XML version 7.6 Bulk processing of XML delivery reports SmsStatusXML operácia),
- requesting RestFul operations (9.4.1 Delivery status of a single message (operation one) or bulk 9.4.3 Bulk SMS delivery status (operation any))
- HTTP(s) method call back - push notification (6.1.2 Callback plain text format or 6.1.3 Callback JSON format).

The methods mentioned above are further explained and described in their respective chapters. More details regarding Callback method are provided below.

6.1.1 Callback delivery report method (Push notification)

A single delivery report is generated upon the Callback request, i.e., a single message or a segment.

The delivery reports is sent to the HTTP(s) address which is pre-defined by the customer. It is assumed that the HTTPs certificate is functional and therefore it is further sent for verification. The customer is fully responsible for the quality and validity of the HTTP certificate.

Upon the customers' request, the Callback address is set up by EuroSMS. The functionality of the address is further verified. The delivery report is in the following format content-type=text/plain or content-type=application/json and it is set up while Callback is being defined by the system. The response format is changeable upon request. This can be completed online.

Callback is called in (roughly) real time after obtaining the information regarding the SMS delivery status from the operator. The customer system may or may not respond to the delivery report receipt. The customer systems' behaviour (expectation) is set up during its implementation. If an acknowledgement of receipt is set and it fails for whatever reason, the SMS API will attempt to send a notification again. The recurrence time is restricted to 24-hour period. After that time has passed, the delivery report becomes undeliverable.

6.1.2 Callback plain text format

Plain text delivery report uses the HTTP GET method and the individual parameters are part of the request.

For an example, please see the following link:

https://callback.zakaznicky.system.com?sent_result=ok&...

***Warning:** The customer systems' Callback line must not use URL redirect, i.e., the address must lead directly to the Callback processing and receiving website.*

6.1.3 Callback JSON format

Callback sends requests with only one delivery_report parameter, which is in the JSON format and contains aspects identical to plain method parameters.

6.1.4 Parameters

Title	Type	Description
sent_result	String	Message sending error code, mostly OK .
sent_time	String	Message sending error code, mostly OK.
delivery_result	String	Message sending error code, mostly OK. 13.2 Full list of SMS, IM delivery states

delivery_time	String	Time the message was delivered to the beneficiary's phone in the yyyy-MM-dd hh:mm:ss format
operator	String	Operator in the MCC.MNC format who owned the telephone number at the time of the message delivery. <i>Warning:</i> does not need to be filled for some destinations (<i>yet</i>).
price	Double	Price of a message/segment shown in EUR.
sms_uuid	String	A unique UUID identifier of a message/segment for messages sent using the SMS API or SMPP method.
segment	Integer	For long messages it is always the segments' serial number. For short messages it is always 1.

6.1.5 JSON delivery report example

```
{
  "sent_result": "OK",
  "sent_time": "2019-01-15 12:20:13",
  "delivery_result": "DELIVRD",
  "delivery_time": "2019-01-15 12:20:22",
  "operator": "231.1",
  "price": 0.031,
  "sms_uuid": "60c525e2-b28a-4e92-9a0f-58dacfa23a86",
  "segment": 1
}
```

6.1.6 Expected response from the customers' system

The format of the customers' system response is redundant as SMS API format can detect this automatically.

The customer system will use the following format to confirm that the delivery report was received and processed:

- plain: ok|sms_uuid
e.g.,: ok|59294546-9F34-4A01-858D-9B27FEB99E2
- JSON:
{
 "sms_uuid": "59294546-9F34-4A01-858D-9B27FEB99E2",
 "status": "ok"
}

6.2 Introduction to received messages

Messages are delivered as a whole, regardless of their length. For long messages, EuroSMS system waits until it receives all of the segments from operators.

Subsequently, the system joins the segments into one long message which is the final message. The final message is then forwarded to the customers system.

The delivery report is in either content-type=text/plain or content-type=application/json format and it is set up while defining the Callback in the system.

The system will attempt to deliver the message to the customer systems within the first 24 after it was delivered. Messages undelivered within the time limit, will be considered undeliverable.

6.2.1 Callback plain text format

Plain text delivery report uses HTTP GET method and its individual parameters are incorporated in the request.

For an example, please see the following link:

https://callback.zakaznický.system.com?&sms_uuid=...

Warning: The customer systems' Callback line must not use URL redirect, i.e., the address must lead directly to the Callback processing and receiving website.

6.2.2 Callback JSON format

Callback fo formáte posiela request s jediným parametrom received, ktorý je v tvare JSON a obsahuje zhodné kľúče ako sú parametre metódy plain.

6.2.3 Parameters sent to Callback

Title	Type	Description
sms_uuid	String	Sms message identifier.
recipient	String	Receiving virtual number.
receive_time	String	Time when message was received from the operator in the following format yyyy-MM-dd hh:mm:ss
sender	String	The senders' phone number.
sms_text	String	Full text of the receiving message.

6.2.4 Expected response from customer systemu

The customer system will use either Plain or Json format to confirm that the delivery report was received:

- plain: ok|sms_uuid
e.g.,: ok|59294546-9F34-4A01-858D-9B27FEBC99E2
- JSON:
 {
 "sms_uuid": "59294546-9F34-4A01-858D-9B27FEBC99E2",
 "status": "ok"
 }

| The format of the customers' system response is redundant as SMS API format can detect this automatically.

7 WebServices Interfaces (2.x version)

7.1 WebServices Interfaces (WS)

The WebServices in the 2.x version are modernised and upgraded calls known from the SMS API version 1.x.

The most prominent changes occurred within the internal processing that are transparent for the clients' system.

The following changes are visible at the interface:

- the digital signature was changed from MD5 to HMAC_SHA1. Furthermore, there was an addition of digitally signed arrays (recipients' original phone number, in the current version, recipients' phone number and text of the message),
- change of a segment identification from Integer to UUID (this is also because of incoming depletion of values of Integer range),
- in order to obtain delivery report for each part of the message, the operations break down long (multi-segment) SMS directly at the input and return identifier for each segment (the original interface 1.x would return a single identifier even long messages),
- a change in the data type of the addressees' (recipients') telephone number from String to Long for the purpose of minimising the input errors,
- added SOAP calls XML,
- system lines up all of the delivery reports in a queue and always provides an additional one. Thus, it is not necessary to request a delivery report for a single segment based on its identifier. Whenever operator acknowledges a status change, the system generates a delivery report and returns it by calling relevant operation,
- operations that were unused have been removed.

7.2 WebService types

We can distinguish between various WS types based on their purpose:

- WS intended for message sending,
- WS intended for information processing used for obtaining delivery status information,
- WS intended for message receiving.

WS (and their respective defining files) are available at the following addresses:

Title	Description
Production environment	https://as.eurosms.com/api/v2/IO
WSDL	https://as.eurosms.com/api/v2/IO?WSDL
XSD 1	https://as.eurosms.com/api/v2/IO?XSD=1
XSD 2	https://as.eurosms.com/api/v2/IO?XSD=2
Testing interface	https://as.eurosms.com/api/v2/IO?tester

7.3 Posielanie jednej SMS volaním parametrami

SmsSendOne operation

7.3.1 The parameters of smsSendOne call

The expected parameters for the SOAP WS calls are in the following order:

Title	Type	Description	Mandatory
Integration ID	String	Lid is a clients' identifier assigned by the provider. Lid together with digital signature are used for authentication. String is in either 1-XXXXXX or 2-YYYYYY format, where X and Y are either numbers or characters. Lid is case sensitive and therefore the character size is essential.	✓
Sender	String	Maximum of 11 alphanumeric characters or phone number in the international format excluding the following symbols; + or 00.	✓
Recipient	Long	The recipients' phone number in the international format or SK/CZ number excluding the country prefixes.	✓
Message	String	The maximum message length is usually 160 characters without diacritical characters or 70 characters including diacritical characters. Long messages are technologically segmented, where the length of each segment is 153 characters, or 67 characters respectively.	✓
Digital signature	String	Digital signature of the <i>Sender</i> , <i>Recipient</i> , and <i>Message</i> arrays. For more details, please see chapter: 4.2 Digital signature calculation	✓
Flags	Integer	Characteristics affecting the messaging behaviour. For more details, please see chapter: 14.1 Flags – characteristics impacting SMS API behaviour	✗
Schedule	String	Message delivery is scheduled to the exact date and time with one-minute accuracy. If the message is dated to the past, it is sent immediately. Supported formats: - dd.MM.yyyy HH:mm - yyyy-MM-dd HH:mm	✗

Based on WSDL, most of the developing environments generate code, that enables calls from the customers' system which is similar to the local functions. See an example from generated clients' Java function:

```
public List<String>
smsSendOne (
    String iid,
    String from,
    long to,
    String msg,
    String signature,
    int flags,
    String schedule)
```

7.3.2 Example of SOAP Request

The following is an example of *SOAP request* that is generated when clients' developing environment does not support automatic generation of call function based on WSDL.

```
<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope
  xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <S:Body xmlns:ns2="http://v2.eurosms.com/">
    <ns2:smsSendOne>
      <iid>1-OXXXXX</iid>
      <from>RZi</from>
      <to>421903622237</to>
      <msg>Test sms</msg>
      <signature>57d8c98ck398d9ikkn8sdf</signature>
      <flags>3</flags>
    </ns2:smsSendOne>
  </S:Body>
</S:Envelope>
```

7.3.3 System response to the **smsSendOne** operation

The system responds with a list of strings, with the minimum length of 2 strings:

- If the *Request* is incorrect (e.g., items are missing), the operation returns an array with length of 2 strings and marks a *Request* error at the position [0], and describes the error at its position [1].
- If the message is accepted by the system, the return array has a minimum of 3 strings.

The meaning of individual array elements:

- The first value refers to the sending status which is listed in 13.1 The list of SMS and IM send/delivery statuses .
- The second value is a verbal description of error,

- Remaining values from index 2 inclusive (calculated from 0) are identifiers in the UUID format of individual SMS segments (parts) originating from broken down message. Such identifiers will be used in the delivery report(7.5 Delivery report processing SmsStatus operation).
If the *Request* is OK, its lenght is at least 3.

7.3.4 An example of system response to **smsSendOne** operation

- For each of the messages that have incorrect digital signatures, there is a return array: ["WRONG_SIGNATURE", "Signature doesnt match."]
- A response for short (or long) message sent strictly via Viber:

```
[  
    "ENQUEUED",  
    "Message accepted and enqueued to send.",  
    "v:08AF040F-CB22-4ED2-91C7-1A787819643B"  
]
```
- A response for long message is split into 3 segments with an attempt to send the message in a standard way or via Viber, based on Viber's reaction, depending on whether the telephone number is registered with Viber. The length of such message can be e.g., 400 characters excluding diacritics:

```
[  
    "ENQUEUED",  
    "Message accepted and enqueued to send.",  
    "v:08AF040F-CB22-4ED2-91C7-1A787819643B",  
    "8B737CBE-3E4D-4D0F-821A-49AC97A7C073",  
    "32AC366A-7973-418C-BDF0-352E2E4D5794",  
    "70FF467D-AFA4-453C-B906-8703DDFD6127"  
]
```
- A response to a long message is split into 3 segments (the length of the message was e.g., 400 characters, excluding diacritical characters):

```
[  
    "ENQUEUED",  
    "Message accepted and enqueued to send.",  
    "8B737CBE-3E4D-4D0F-821A-49AC97A7C073",  
    "32AC366A-7973-418C-BDF0-352E2E4D5794",  
    "70FF467D-AFA4-453C-B906-8703DDFD6127"  
]
```

7.3.5 An example of *SOAP Response*

The following is a *SOAP Response* which is a response to incorrect digital signature:

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope  
  xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"  
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">  
  <SOAP-ENV:Header/>  
  <S:Body xmlns:ns2="http://v2.eurosms.com/">  
    <ns2:smsSendOneResponse>  
      <return>WRONG_SIGNATURE</return>  
      <return>Signature doesnt match.</return>  
    </ns2:smsSendOneResponse>  
  </S:Body>  
</S:Envelope>
```

7.4 Sending a single SMS in XML format

SmsSendOneXML operation

The **smsSendOneXML** operation is almost identical to the **smsSendOne**. In contrast to **smsSendOne**, **smsSendOneXML** has a different input format. The advantage of using the **smsSendOneXML** is that in automatically generated clients' calls as the clients' development environment directly creates objects for XML inputs based on the WSDL objects.

7.4.1 XML format

Basic (root) XML `<sms>...</sms>` element contains the following attributes and elements:

7.4.2 Attributes

Title	Type	Description	Mandatory
iid	String	Iid is an clients' identifier assigned to them by a provider. Together with digital signature, iid is used for authentication. String is either in the 1-XXXXXX or 2-YYYYYY format, where X and Y are numbers or characters. Iid is case sensitive.	✓
signature	String	Digital signature of the <i>Sender</i> , <i>Recipient</i> , and <i>Message</i> arrays. For more details, please see chapter: 4.2 Digital signature calculation	✓
flags	Integer	Characteristics that affect the messaging behaviour. For more details, please see chapter: 14.1 Flags – characteristics impacting SMS API behaviour	✗
schedule	String	Message delivery is scheduled to the exact date and time with one-minute accuracy. If the message is dated to the past it is sent immediately. Supported formats: - dd.MM.yyyy HH:mm - yyyy-MM-dd HH:mm	✗

7.4.3 Elements

Title	Type	Description	Mandatory
from	String	Maximum of 11 alphanumeric characters or telephone number in the international format, excluding characters such as + or 00.	✓
to	Long	Maximum of 11 alphanumeric characters or telephone number in the international format, excluding characters	✓

		such as + or 00.	
message	String	The length of the message is usually 160 characters excluding diacritical characters, or 70 including diacritical characters. Long messages are technologically segmented, where the length of the segment is 153 or 67 characters, respectively.	✓

7.4.4 An example of XML Request

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<sms flags="3"
      iid="1-OLX3XX"
      schedule="2018-11-25 13:27:15"
      signature="7d78f038c87f771759f7fffcad296a04dfabd53a">
  <sender>RZi</sender>
  <recipient>421902022000</recipient>
  <msg>7) Special: | ^ € { } [ ~ ] \ @US 2018-11-25 13:27:15</msg>
</sms>
```

7.4.5 System response to smsSendOneXML operation

The operation corresponds to XML objects with a <result>..</result> basic element and strictly contains only elements.

7.4.6 XML response elements

Title	Type	Description	Mandatory
err	String	Error identifiers listen in 13.1 The list of SMS and IM send/delivery statuses .	✓
err_desc	String	Verbal description of error.	✓
uuids	Element	<uuid>String</uuid> Each of the element of <uuid> type acts as an identifier for each segment. Messages approved to be sent, contain at least one <uuid> element. Warning: The element <uuid> in messages sent to Viber will contain prefix "in:." Remaining <uuid> elements without prefix will correspond to an alternative sending using the standard SMS route. The delivery report will be returned either to Viber (if the telephone number has an active Viber), or to remaining <uuid> elements (if the SMS was sent by using the standard route). Example: <uuid>v:7E4872B4-60DE-4657-9EC8-E5508D8F9180</uuid>	✗

7.4.7 Example of XML Response

The following example is an example of long a message that was technologically split into 2 segments:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<result>
  <err>ENQUEUED</err>
  <errDesc>Message accepted and enqueued to send</errDesc>
  <uuids>
    <uuid>77c8078f-6cc4-4671-8639-0cef646478b3</uuid>
    <uuid>e2c80cbd-04b6-4717-bdbb-bc6542b96bca</uuid>
  </uuids>
  <segments>2</segments>
</result>
```

7.5 Delivery report processing

SmsStatus operation

SMS API sends delivery reports that are placed in a queue. The **smsStatus** operation returns the status of one or more sms/segments. EuroSMS systems collect information regarding the delivery reports statuses and forwards the oldest one on request. The operation can be called repeatedly, until it returns the information that the queue has been emptied (NoMoreDLRs error).

The information regarding the messages/segments statuses is *never* sent again using the same identifier (see 13.1 The list of SMS and IM send/delivery statuses).

7.5.1 SmsStatus operation parameters

Title	Type	Description	Mandatory
iid	String	Integration identifier.	✓
trxid	String	Any string of characters or a number that identifies this transaction, eg., UUID.	✓
count	Integer	The number of delivery reports, how many does customer service queries at once. The minimum of 1 and the maximum of 128.	✓
signature	String	TransactionId digital signature.	✓

7.5.2 Return values

Operation will return a String (String[[[]]) version of all elements of a 2D array in which one row has the following structure:

Position	Description
[0]	Internal status of the delivery report, ok by default. If there are no more delivery report available, system returns one row with the NoMoreDLRs status.
[1]	Message/segment identifier obtained after smsSendOne or smsSendOneXML was sent.
[2]	Message delivery status is specified in the chapter listed below; 13.1 The list of SMS and IM send/delivery statuses
[3]	Date and time the message was sent in the dd.MM.rrrr mm:hh:ss format
[4]	Date and time delivery status changes are in the dd.MM.rrrr mm:hh:ss format
[5]	Serial number of a segment. For short messages this is always 1.
[6]	Price of a message/segment.
[7]	MCC.MNC is network identifier, where the telephone number belongs. E.g., Orange.sk 231.1 , Telekom.sk 231.2 , 4ka 231.3 , O2 231.6 , etc.

7.5.3 An example of a reurn array

SMS API returns an array with a length of 3. That is because 3 delivery reports were requested or because none of the delivery reports were available.

```
[  
  [ok, c8c599a1-8b3a-497e-ba16-609c12e1bb63, DELIVRD,  
    25.11.2018 13:19:58, 25.11.2018 13:19:58, 1, 0.033, 231.2],  
  
  [ok, 964786e2-c113-466f-bbdf-6d69cb0d2470, DELIVRD,  
    25.11.2018 13:19:58, 25.11.2018 13:19:58, 1, 0.033, 231.2],  
  
  [ok, c8c599a1-8b3a-497e-ba16-609c12e1bb63, DELIVRD,  
    25.11.2018 13:19:58, 25.11.2018 13:19:58, 1, 0.033, 231.2]  
]
```

7.6 Bulk processing of XML delivery reports

SmsStatusXML operácia

The **smsStatusXML** operation is almost identical to **smsStatus**. In contrast **smsStatus**, **smsStatusXMLs**' call-input and output is XML.

The input parameters (elements) of the selective XML are identical 7.5 Delivery report processing **SmsStatus** operation.

7.6.1 An example of XML Request

Signed as *trxid*.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>  
<DLRRequest>  
  <iid>2-XPoA34</iid>
```

```
<trxid>A408925E-B2B7-46C4-91A2-FBBA22B9716C</trxid>
<count>128</count>
<signature>6f56060b6b7db97ca25782b771cca0a65077bd5b</signature>
</DLRRequest>
```

7.6.2 XML Response

SMS API responds with a XML document in the following format, where the root element is `<DeliveryReports>...</DeliveryReports>`

Element title	Type	Description
status	String	Internal status of the delivery report, ok by default. If there are no more delivery reports available, system returns one row with the status of NoMoreDLRs , count has value of 0.
count	Integer	Number of returned delivery reports, maximum of 128.
dlrs	XML	XML structure of dlr objects, where each object is one delivery report. For more details, please see the table below.

Následující tabulka obsahuje elementy XML objektu **dlr**.

Element title	Type	Description
smsUUID	String	A SMS/segment identifier obtained during the process of message sending.
sendStatus	String	Value obtained from the table: 13.1 The list of SMS and IM send/delivery statuses
sendTime	String	Time the message was sent in the yyyy-MM-dd hh:mm:ss format
deliveryStatus	String	Value obtained from the table: 13.2 Full list of SMS, IM delivery states
deliveryTime	String	Delivery time of a status changes in the yyyy-MM-dd hh:mm:ss format
operator	String	MCC.MNC is network identifier, where the phone number belongs. For example, Orange.sk 231.1 , Telekom.sk 231.2 , 4ka 231.3 , O2 231.6 , ...
segment	Integer	Serial number of a segment for long messages. For short messages it is always 1.
price	Double	Price of SMS or a segment corresponding to a long message.

7.6.3 An example of XML Response

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<DeliveryReports>
  <status>ok</status>
  <count>2</count>
  <dlrs>
```

```
<dlr>
  <smsUUID>b9431b26-0869-4e24-97d2-dcf18628a68e</smsUUID>
  <sendStatus>ok</sendStatus>
  <sendTime>26.11.2018 17:37:09</sendTime>
  <deliveryStatus>DELIVRD</deliveryStatus>
  <deliveryTime>26.11.2018 17:37:08</deliveryTime>
  <operator>231.2</operator>
  <segment>1</segment>
  <price>0.033</price>
</dlr>
<dlr>
  <smsUUID>e2c80cbd-04b6-4717-bdbb-bc6542b96bca</smsUUID>
  <sendStatus>ok</sendStatus>
  <sendTime>26.11.2018 17:37:09</sendTime>
  <deliveryStatus>DELIVRD</deliveryStatus>
  <deliveryTime>26.11.2018 17:37:08</deliveryTime>
  <operator>231.2</operator>
  <segment>1</segment>
  <price>0.033</price>
</dlr>
</dlrs>
</DeliveryReports>
```

7.7 Reception of a single message

SmsReceiveOne operation

The operation is used to select one message from the ones received by the system.

Received messages are sent from a mobile device (charged by the operators' fee) to a virtual number either from Telekom a.s. or O2 Slovakia s.r.o. Virtual numbers are assigned only to the customers on their request. Received messages are not accessible to other EuroSMS customers. Virtual numbers are NOT charged per message. Instead, virtual numbers are charged at the flat rate.

Virtual numbers are not callable. They are used exclusively to receive SMS.

SMS received on the virtual numbers are stored in the EuroSMS systems. They can be published on request by using the following strategies:

- available at www.eurosms.com,
- publishable by e-mail (simultaneously to multiple addresses), see 5.1.4 ReceiveByMail method,
- using the SMPP protocol,
- by calling the Callback customers' line, see 6.1.1 Callback delivery report method (Push notification),
- published by calling SMS API operation (for more detail see the chapter below).

7.7.1 Two-way messaging

Upon request, it is possible to set up an automated reply to the receiving number. This is referred to as two-way SMS messaging.

Please contact support@eurosms.com to put in a request for an automated response to the assigned virtual number. support@eurosms.com.

7.7.2 Calling **smsReceiveOne** operation

Using Java, the operation is callable by three mandatory parameters. The generated clients' declaration can potentially look like this:

```
private static java.util.List<java.lang.String>
    smsReceiveOne(
        String iid,
        long to,
        String signature )
```

The parameter *to* is digitally signed (virtual number).

The method returns a string value, where the status of the response is stored at position [0]

- ok- this is the only message status that confirms that message has been correctly received, followed by the message values (described in the paragraph below),
- the following refers to one of the statuses that describes incorrect input parameters: WRONG_KEY, NO_SGN, UNKNOWN_RECIPIENT, WRONG_IID, WRONG_SIGNATURE. The statuses are described in 7.7.3 Arrays of returning values of received messages),
- jeden zo stavov popisujúcich chybné vstupné parametre: WRONG_KEY, NO_SGN, UNKNOWN_RECIPIENT, WRONG_IID, WRONG_SIGNATURE. Stavý sú popísané v 13.1 The list of SMS and IM send/delivery statuses . VIn this case, position [1] contains detailed text description of the error,
- NO_MORE_MESSAGES status, suggests that systems currently stores no record of any unprocessed received message on the given recipients' number.

7.7.3 Arrays of returning values of received messages

The following table describes arrays of returning values of short message (length of 5):

Position	Description
[0]	Status "ok"
[1]	UUID of received message
[2]	Message sender (telephone number)
[3]	Message/segment text transcoded from any character set into UTF8
[4]	Message delivery time in the rrrr-MM-dd hh:mm:ss format

For segmented (long) messages the length of returning array is 8. Other values describe each of the segments in a way that allows the customer system to join them into the final message:

Position	Description
[5]	The connection identifier is identical for all segments, it can also take up a negative number <i>int</i> .
[6]	The total number of segments (always >1).
[7]	Segment serial number calculated from 1. Allowed values: 1, 2,3... Technological maximum is 255.

7.7.4 An example of short messages received by beneficiary

```
[  
  "ok",  
  "ebdb4ffc-77b0-4b73-9a4d-280d6364b983",  
  "421903622273",  
  "7) Special: | ^ € { } [ ~ ] \ @US 2018-12-03 11:27:31",  
  "2018-12-03 11:27:32"  
]
```

7.7.5 An example of received message segment belonging to a long message

```
[  
  "ok",  
  "ebdb4ffc-77b0-4b73-9a4d-280d6364b983",  
  "421903622273",  
  "6) € TEST 10, TEST 10, TEST 10, TEST 10, TEST 10, TEST 10,  
    TEST 10, TEST 10, TEST 10, TEST 10, TEST 10, TEST 10,  
    TEST 10, TEST 10, TEST 10, TEST 10, TES"  
  "2018-12-03 11:27:32",  
  "-21",  
  "3",  
  "1"  
]
```

7.8 Receiving bulk SMS via XML

SmsReceiveMoreXML operation

In contrast to **smsReceiveOne** operation, **smsReceiveMoreXML** is more suitable for

receiving higher number of incoming messages by a single SMS API call. Additionally, it is suitable for receiving structured XML-based objects.

7.8.1 Parameters of **smsReceiveMoreXML** calls

The parameters for SOAP WS calls are expected in the following order:

Title	Type	Description	Mandatory
iid	String	<i>Iid</i> is a clients' identifier assigned to them by a provider. Together with digital signature, iid is used for authentication. String is in the 1-XXXXXX or 2-YYYYYY format, where X and Y are numbers or characters. Iid is case sensitive.	✓
receiverNumber	long	Phone number for receiving messages associated with <i>integrationId</i> in the international format e.g., 421902022000.	✓
count	int	The expected maximum number of receiving SMS/segments that were returned by the operation for a single call. Maximum value of 32, minimum of 1.	✓
signature	String	Digital signature of the <i>receiverNumber</i> array. For more details see chapter; 4.2 Digital signature calculation	✓

7.8.2 Format prototype of the generated **smsReceiveMoreXML** function

```
private static XMLReceivedResponseV2  
    smsReceiveMoreXML(  
        String iid,  
        long receiverNumber,  
        int count,  
        String signature )
```

7.8.3 Response of **smsReceiveMoreXML** operation

SMS API responds with a XML object. The given example displays 1.) a message (of a standard length), and 2.) a single segment belonging to a long message.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>  
<ReceivedMessages>  
  <status>ok</status>  
  <count>2</count>  
  <rcvds>  
    <rcvd>  
      <msg>  
        7) Special: | ^ € { } [ ~ ] \ @US 2018-12-07 18:05:46
```

```
</msg>
<receivedTime>2018-12-07 18:05:48</receivedTime>
<recipient>421902022000</recipient>
<sender>RZi</sender>
<uuid>8b41e680-4938-425e-88b8-b0b5fbe793bf</uuid>
</rcvd>
<rcvd>
  <segmentation>
    <segmentNum>1</segmentNum>
    <segmentCount>2</segmentCount>
    <segmentID>18</segmentID>
  </segmentation>
  <msg>
    6) € TEST 10, TEST 10, TEST 10, TEST 10, TEST 10, TEST 10,
    TEST 10, TEST 10, TEST 10, TEST 10, TEST 10, TEST 10, TEST 10,
    TEST 10, TEST 10, TEST 10, TES</msg>
  <receivedTime>2018-12-07 18:05:48</receivedTime>
  <recipient>421902022000</recipient>
  <sender>RZi</sender>
  <uuid>95ca665b-de8f-40d4-b65d-bfc5be02250c</uuid>
</rcvd>
</rcvds>
</ReceivedMessages>
```

7.8.4 XML elements of the **root** object

Root element of a response is `<ReceivedMessages>...</ReceivedMessages>`.

Each root element contains the following elements:

Title	Type	Description	Mandatory
status	String	Described in 7.7.2 Calling smsReceiveOne operation	✓
count	int	Number of returned messages/segments	✓
rcvds	XML	Number of returned messages. If the status is anything other than ok alebo <i>count</i> =0, it does not contain any XML objects. If the status is ok and <i>count</i> >0, it contains XML objects of rcvd type (described below).	✓

7.8.5 XML element **rcvd**

Rcvd element contains all information regarding the received message.

A received segment contains information regarding the segmentation. This information is then used for the purpose of joining the received messages into the final message.

When it is not necessary to use this method of receiving messages, it is recommended to use callback method instead (6.2 Introduction to received

messages). The callback method delivers messages in their complete form, i.e., uses a single notification even for long messages (messages split into multiple segments).

Title	Type	Description	Mandatory
msg	String	Text of the received message/segment.	✓
receivedTime	String	Date and time of when the message was received in the yyyy-MM-dd hh:mm:ss format.	✓
recipient	String	Receiving phone number.	✓
sender	String	Recipients' telephone number or the text of the message.	✓
uuid	String	A unique identifier of a message/segment.	✓
segmentation	XML	XML object containing information that is essential for segment joining. For short messages it is meaningless and thus it does not contain any values.	✗

7.8.6 XML element **segmentation**

The element **segmentation** contains information that is used for the purpose of joining segments into long SMS.

This information is stored and relevant only for long messages.

Segmentation is set up by the operator and it contains data from so-called SAR (Segmentation and Reassembly), for more information, please see 3.2 The fundamentals of Short Message Services and Segmentation.

Title	Type	Description	Mandatory
segmentID	int	Identifier. All segments belonging to a single message have identical identifier.	✓
segmentCount	int	Number of segments corresponding to a single received message. E.g., if a message contains 3 segments, <i>segmentCount</i> =3, 3segments with identical <i>segmentID</i> will be delivered.	✓
segmentNum	int	Segments' serial number.	✓

8 HTTPs interfaces (ver. 2.x)

HTTPs interface represents the most straightforward SMS API communication variant. Both the GET and the POST method can be used.

Warning: all parameters must be URL encoded.

Example: consider sending parameters with value of „řčř test of diacritical characters“. In order to correctly process such message, the message must be encoded accordingly:

<https://xx.xx.xx/Sent?msg=Test+diakritiky+%C4%BE%C5%A1%C4%8D%C5%A5%C5%A5>

The functionality of the HTTPs interface was built on the same principles as WebServices. Therefore, the titles, types, and all the parameter values are identical and will not be further discussed. However, they will be referenced in WebServices, as WebServices contain detailed description of them.

The production interface is available at:

<https://as.eurosms.com/api/v2/Sender>

8.1 Parameters sent in Request

Decisive and mandatory sending parameter:

Title	Type	Description	Mandatory
action	String	Can take up the following values: <i>send1SMS</i> <i>send1SMSHTTP</i> <i>statusSMS</i> <i>status1SMSHTTP</i> <i>receiveSMS</i> <i>receive1SMSHTTP</i> Two special values: <i>validate1SMS</i> <i>validate1SMSHTTP</i>	✓

The parameter meaning listed by prefix:

- send* – used to send a message,
- status* – used to determine the delivery status,
- receive* – used to retrieve received messages,
- validate* – equivalent to send, however, messages are only syntactically and semantically reviewed and they are not being physically send to the phone

number.

The HTTP postfix suggests that parameters will be further named ([see](#)). The expected XML parameter for absent HTTP is described below.

8.2 List of parameters used for SMS sending

It is recommended to send HTTP using *GET* method.

Title	Type	Description	Mandatory
action	String	One of the <i>send1SMSHTTP</i> or <i>validate1SMSHTTP</i> values. For <i>validate1SMSHTTP</i> value, the message is only validated syntactically. Thus, the message will not be send.	✓
iid	String	Iid is an clients' identifier assigned to them by a provider. Together with digital signature, iid is used for authentication. String is in the 1-XXXXXX or 2-YYYYYY format, where X and Y are numbers or characters. Iid is case sensitive.	✓
signature	String	Digital signature corresponding to the <i>Sender</i> , <i>Receiver</i> , <i>Message</i> arrays. For more details, please see chapter 4.2 Digital signature calculation	✓
flags	String	Characteristics impacting behaviour of the message sending. For more details, see 14.1 Flags – characteristics impacting SMS API behaviour	✗
schedule	String	Message delivery is scheduled to the exact date and time with one-minute accuracy. If the message is dated to the past it is sent immediately. Supported formats: • dd.MM.yyyy HH:mm • yyyy-MM-dd HH:mm	✗
from	String	Maximum of 11 alphanumeric characters or telephone number in the international format, excluding + or 00 characters.	✓
to	String	Receivers' telephone number in the international or SK/CZ format excluding prefix.	✓
message or msg	String	The length of the message is usually 160 characters excluding diacritical characters, or 70 including diacritical characters. Long messages are technologically segmented, where the length of each segment is 153, or 67 characters respectively.	✓

An example (*i* and *signature* is not real data):

<https://as.eurosms.com/api/v2/Sender>
?action=validate1SMSHTTP
&iid=1-XYXYXY
&from=RZi
&to=421903622237
&message=Test+message
&signature=768668ds87ads876d8768adsd
&flags=3
&schedule=2010-01-18+13%3A30

8.2.1 System response

SMS API responds with a document (webpage) in the text format (Content-type: text/plain) with a single row, where parameters are separated by the “|” character.

Position	Description	Mandatory
[0]	Error code from the table 13.1 The list of SMS and IM send/delivery statuses	✓
[1]	Error containing text description.	✓
[2]...[x]	Requests that are evaluated by the system and considered to be correct and therefore accepted. Such requests have UUID identifiers for each segment at the position [2] and further. For short and accepted messages, the message string has minimum of 3 parameters. <i>Warning:</i> SMS that are sent to Viber contain a single identifier with v prefix. An example: “v:7E4872B4-60DE-4657-9EC8-E5508D8F9180” For more information regarding Viber, please see; 7.4.6 XML response elements	✗

8.2.2 Example of *Plain* response

Parameters of the response are separated by the “|” character:

- incorrect *Request* (message not sent):
UNKNOWN_OPERATION|Unknown operation or missing action param.
- correct *Request* (short message, sent):
ENQUEUED|Message accepted and enqueued to send|1|588f02fe-dec4-4ada-b948-aab2e320dcbc
- correct *Request* (long message, 2 segments, message sent):
ENQUEUED|Message accepted and enqueued to send|2|889a4e59-ec80-429d-82ce-f7dd0459bbd2|78efac19-b293-4b56-be30-e169f3f62f4d
- valid *Request* (long message, 2 segments, the message is not sent):
VALID_REQUEST|The request is valid. Message wasn't send.|2|

FAKE-90aa64c5-700d-41b9-ad2f-73cd08a1b15e | FAKE-5e01b4ef-0365-4b08-9ed1-17350ff87e63

8.3 Messages sent in XML format

It is recommended to send HTTP using the *POST* method.

Messages sent in XML format have 2 parameters:

Title	Type	Description	Mandatory
action	String	One of the <i>send1SMS</i> or <i>validate1SMS</i> values. <i>Validate1SMS</i> message is only syntactically validated and will not be sent	✓
xml	String	XML parameter is identical to WebServices and is further described in 7.4 Sending a single SMS in XML format SmsSendOneXML operation.	✓

Validate1SMS message will not be sent, and the SMS API response will be as usual.

8.3.1 System response to the messages sent in XML format

SMS API responds with XML document (Content-type: text/xml). The XML format and its elements are described in 7.4.5 System response to smsSendOneXML operation.

8.3.2 Examples of XML response

Positive response to *action=validate1SMS*

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<result>
  <uuids>
    <uuid>FAKE-4d430426-5788-4b59-88ac-9b5824651e04</uuid>
    <uuid>FAKE-aeebef75-85c7-408b-9525-f48c89bd4cef</uuid>
  </uuids>
  <err>VALID_REQUEST</err>
  <errDesc>The request is valid. Message wasn't send.</errDesc>
  <segments>2</segments>
</result>
```

Error response to *action=send1SMS*

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<result>
  <err>WRONG_XML_FORMAT</err>
```



```
<errDesc>XML is unparsable, wrong format.</errDesc>
<segments>0</segments>
</result>
```

8.4 Requesting *Plain* delivery report

It is recommended to send HTTP using *GET* method.

By a single call, system responds using an information obtained from the delivery report. The information obtained from the delivery report is separated by the “|” character.

Title	Type	Description	Mandatory
action	String	<i>statusSMSHTTP</i>	✓
iid	String	Integration identifier.	✓
trxid	String	Any character or number string that identifies this transaction, e.g., UUID.	✓
signature	String	Digital signature transactionId.	✓

8.4.1 Odpoveď na vyžiadanú doručenkú typu *Plain*

The content-type=text/plain page format contains arrays that are listed in the table below. The array order is fixed.

#	Title	Type	Description
1	smsUUID	String	SMS/segment identifier obtained during the process of message sending.
2	sendStatus	String	Value obtained from table 13.1 The list of SMS and IM send/delivery statuses
3	sendTime	String	Time the message was sent in the yyyy-MM-dd hh:mm:ss format.
4	deliveryStatus	String	Value obtained from table 13.2 Full list of SMS, IM delivery states
5	deliveryTime	String	Delivery time of a status change in yyyy-MM-dd hh:mm:ss format.
6	operator	String	MCC.MNC is network identifier of where the phone number belongs. For example, Orange.sk 231.1 , Telekom.sk 231.2 , 4ka 231.3 , O2 231.6 , ...
7	segment	Integer	Serial number of a segment belonging to a long message. The value is always 1 for short messages .
8	price	Double	Price of SMS or a segment.

8.4.2 An example of *Plain* delivery report

The following example displays delivery report in its final DELIVRD stage (message was received).

```
b9431b26-0869-4e24-97d2-dcf18628a68e |
ok|26.11.2018 17:37:09|DELIVRD|26.11.2018 17:37:08|231.2|1|0.033
```

8.5 XML delivery report(s) request

It is recommended to send HTTP using *POST* method.

Upon request, customers can receive multiple delivery reports by a single call.

Request contains two parameters that are listed in the following table.

Title	Type	Description
action	String	<i>statusSMS</i>
xml	String	XML object described below.

8.5.1 Formát XML pre vyžiadanie doručeníek

The name of the root element is `<DLRRRequest>....</DLRRRequest>` and it contains the following elements:

Title	Type	Description	Mandatory
iid	String	Integration Id.	✓
trxid	String	An arbitrary transaction identifier, e.g., UUID.	✓
signature	String	Digital signature <i>trxid</i> .	✓
count	Integer	Number of delivery reports with minimum of 1 and maximum of 100 reports. If not specified, the default value is 5.	✗

8.5.2 An example of XML requesting delivery reports

An example of delivery report, <i>action=statusSMS</i>
<pre><?xml version="1.0" encoding="UTF-8" standalone="yes"?> <DLRRRequest> <iid>2-XDP0ZZ</iid> <trxid>d04740c3-7e16-4d37-b093-bd3e22d6d3c2</trxid> <signature>768668ds87ads876d8768adsd</signature> <count>20</count> </DLRRRequest></pre>

8.5.3 XML response to the delivery report request

SMS API responds with a xml formatted webpage, content-type=text/xml. The parameters are displayed in 7.6.2 XML Response . Examples are shown and further discussed in the 7.6.3 An example of XML Response paragraph.

8.6 Received message of *Plain* request type

The GET method is recommended for HTTP. The use of Plain method allows for reception of a single message. The details and principles that guide this method are identical to WebService which is described in 7.7 Reception of a single message SmsReceiveOne operation. The required parameter value is action=*receiveSMSHTTP*.

8.6.1 An example of a single call

An example (*iid* and *signature* are not based on real data):

```
https://as.eurosms.com/api/v2/Sender  
?action=receiveSMSHTTP  
&iid=1-XYXYXY  
&to=421902112233  
&signature=768668ds87ads876d8768adsd
```

8.6.2 System response

The system responds with a content-type=text/plain formatted webpage containing a single row.

The individual parameters are separated by the “|” character. The return values are described in the 7.7.3 Arrays of returning values of received messages chapter.

8.6.3 An example of a response to a message received in the *Plain* format

```
ok|ebdb4ffc-77b0-4b73-9a4d-280d6364b983|421903622273|7) Special: ^ €  
{ } [ ~ ] \ @US 2018-12-03 11:27:31|2018-12-03 11:27:32
```

8.7 Received messages - XML Request type

It is recommended to use POST method for HTTP. Using this method allows for receiving multiple messages simultaneously.

The details and principles that guide this method are identical to WebService which is described in 7.8 Receiving bulk SMS via XML SmsReceiveMoreXML operation.

Title	Type	Description
action	String	<i>receiveSMS</i>
xml	String	XML object described in 8.7.1.

8.7.1 XML formatted request for received messages

The name of root element is `<ReceivedRequest>...</ReceivedRequest>` and it contains the following elements:

Title	Type	Description	Mandatory
iid	String	Integration Id.	✓
to	String	Recipients' phone number.	✓
signature	String	Digital signature of the to element.	✓
count	Integer	Number of delivery reports with minimum of 1 and maximum of 100 reports. If not specified, the default value is 5.	✗

8.7.2 An example of XML object used for message reception

An example of requesting message reception , *action=receiveSMS*

```
<?xml version="1.0" encoding="UTF-8"?>
<ReceivedRequest>
  <iid>1-XXXXYY</iid>
  <to>421902999999</to>
  <count>2</count>
  <signature>d5d45811809d57087689eeee1a5702834fd5becc</signature>
</ReceivedRequest>
```

8.7.3 XML system response

System responds with the content-type=text/xml formatted webpage which is further described in the 7.8.3 Response of smsReceiveMoreXML operation chapter.

9 RestFul (ver. 3.x) interfaces

9.1 Principals

Customer system sends *JSON Request* to SMS API.

Subsequently, SMS API responds with *JSON object*. *JSON request* is then sent to backend system, which ensures distribution of the messages to the operators.

Message delivery and sending statuses are available by a calling to SMS API. More specifically, they are available at the www.eurosms.com portal, or by CallBack (Push) method which is further described in 6.1.1 CallBack delivery report method (Push notification) chapter.

JSON object of the *Request* type can contain 3 distinct aggregations of messages:

- Single – a *single* message.
- Multi (*bulk*) –multiple messages with various texts and recipients.
- Multi recipient (*group*) – same text sent to group of recipients.

Based on the requested response type, JSON object of the *Response* type can contain the following:

- A simple information regarding the *Request* status and the number of accepted/rejected messages.
- Complex information corresponding to each of the messages- status and message uuid of the message located in the system.

9.1.1 Web Application Description Language (WADL)

To obtain the definition of the Restful API operations for the production version, please see:

- <https://as.eurosms.com/api/v3/application.wadl>
- <https://as.eurosms.com/api/v3/application.wadl?detail>

9.2 JSON

Upozornenie: : The following example contains all variants of JSON values, however, this cannot be sent nor used in the format shown below. Each of the Restful interface variants (– **one**, **o2m**, **m2m**) are used for different message sending methods and are further discussed in this document.

An abstract example of JSON Request object.

```
{
  "iid": "1-ABCDEF",
  "dflgs": 2,
  "rsp": "basic",
  "dsndr": "Info",
  "start": "2018-01-10 12:30",
  "end": "2018-01-10 16:30",
  "dur": "12:00",

  "msg": {
    "rcpt": 421903622237,
    "txt": "Jedna sprava na jedno cislo",
    "flgs": 3,
    "sgn": "dfsdf244323",
    "sndr": "RoboZilka"
  },

  "msgs": [
    {
      "txt": "Jedna sprava viacerym odosielateľom.",
      "flgs": 3,
      "sgn": "76656sfdgdg",
      "rcpts": [
        903622237,
        420766121212
      ],
      "sndr": "RoboZilka"
    },
    {
      "rcpt": {
        "r": 4412345678,
        "f": "21C334C7-CD7E-45D7-AA88-FC9C58A7FF47"
      },
      "txt": "Jedna sprava na ine cislo",
      "sgn": "dfsdf244323"
    }
  ]
}
```

9.2.1 Request JSON **root** object

Abbrevation	Title	Type	Description	Mandatory
iid	Integration ID	String	Iid is an clients' identifier assigned to them by their provider. Together with digital signature, iid is used for authentication. String is in the 1-XXXXXX or 2-YYYYYY format, where X and Y are numbers or characters.	✓
dflgs	Default flags	int	Flags have an impact on the message parameters. Default flags are used when no flags were found in JSON, i.e., m2m . <i>Default: 2</i>	✗
rsp	Response format	String	An optional parameter. Depending on the API value, it returns either the number of received and rejected messages, or complete information. <i>Default: basic</i> Hodnoty: <i>basic</i> - returns only estimated number of the received/rejected messages and their statuses. <i>full</i> - full information attached to each message sent. For more details about the system response, please see chapter: „Response JSON Objekty“	✗
dsndr	Default sender	String	Two variants: <i>Numerical</i> - telephone number in its full format (e.g., 421903622237, character + is unnecessary and thus will be eliminated), <i>Textový</i> sender, maximum of 11 letters and numbers. Other supported characters include: hyphen, space, and period. It is not recommended to use other characters as using such characters might result in failing to send the entire message.	✓
msg	A single message	JSON Object	Contains a single message. The message can be sent to one or multiple recipients.	*
msgs	Multiple messages	JSON Array	Message array in the <code>msg</code> format.	*
start	Beginning of SMS	String	<code>start</code> and <code>dur</code> objects are used to set up a delivery time. If not specified, messages are	✗

	sending		sent immediately. The time and date are in the following format: yyyy-MM-dd hh:mm. If <code>dur</code> is not specified, messages are sent continuously.	
<code>dur</code>	Sending duration	String	The time interval. Format: "hh:mm". Example: • number of messages sent: 11, • delivery time (immediately), e.g., 13:10, • <code>dur="1:00"</code>, • SMS are sent as follows: ◦ 13:10, 13:16, 13:22, 13:28, 13:34, 13:40, 13:46, 13:52, 13:58, 14:04, 14:10	✗

* JSON must contain at least one of the `msg`/`msgs` objects. The use of these objects is dependent on the Restful operation used.

9.2.2 JSON object of the *msg* type

Abbreviation	Title	Type	Description	Mandatory
sndr	Sender	String	Format as dsndr.	*
rcpt	Recipient	Long or JSON	Recipient (addressee) is a telephone number presented in its full format, i.e., includes prefix belonging to its respective country. For example, 420766121212. For Czech and Slovak phone numbers only, it is allowed to use the short number format, i.e., CZ: 766121212, SK: 903622237. These will be detected and further adjusted into to required international format.	**
rcpts	List of Recipients	Long[] or JSON[]	JSON addressee array in one of the rcpt formats.	**
txt	Text	String	Text with the minimum length of 1 character. UTF-8 text encoding system. The allowed message lengths and the diacritical characters are further explained in flgs.	✓
sgn	Digital signature	String	Digital signature will be calculated by using the <i>integration id</i> , <i>senders and recipients telephone number</i> , and the <i>SMS text message</i> . Example of signature calculation for a single recipient Sender: RZi , addressee: 421903622237 , sms text: Testing message , integration key: Gh-s7-J6 : • sender, addressee, and a message will be joined into a single String: RZi421903622237Testing message • HMAC_SHA1 algorithm and a key Gh-s7-J6 will be applied: 6f56060b6b7db97ca25782b771cca0a65077bd5b the above is referred to as the digital signature If rcpts is used, the digital signature calculated from the whole array. Addressees' phone numbers with their integration keys will be joined together in the order in which they were listed. Example of signature calculation for number of	✓

			recipients Sender: <i>RZi</i>, addressee: [<i>421903622237</i>, <i>420766121212</i>], sms text: <i>Testing message</i>, integration key: <i>Gh-s7-J6</i>: • <i>RZi421903622237420766121212</i>Testing message • Signature: <i>fa02681d79dfe591300a7f181837a58d630be5ad</i>	
flgs	Flags		Flags have an impact on API behaviour and in the message length and diacritical character use: • 0x02 – allows to send messages that are longer than 160 ascii characters, • 0x04 –allows for diacritical characters. The length of the message shortens to the maximum of 70 characters. flgs not set to the values of 0x02 and ascii message exceeding the maximum of 160 characters will result in rejecting the message. Therefore, such message cannot be sent. The same rule applies to the messages that are marked as flgs=0x04. If such message exceeds 70 characters, it will be rejected and not sent. Flags can be combined, long message with diacritical characters: flgs=0x06. Message length based on the flgs setting: • if not set up, the value of 0 is used. The message is then ascii coded with the maximum of 160 characters, • flgs=0x02 – the message will be counted and invoiced as n x 153 characters (segments), where n operators do not recommend more than 4. Thus, the message contains no more than 612 characters, • flgs=0x04 – message including diacritical characters (řšťžýá....) will be sent. The maximum length of such message is 70 characters, • flgs=0x06 – message length with diacritical characters, n x 67, recommended maximum length of 268 characters.	***

			SMS API is not limiting to the number of segments. To see the full list of FLAGS: 14.1 Flags – characteristics impacting SMS API behaviour	
--	--	--	--	--

* If `dsndr` nor `sndr` is specified, the message will be considered to lack essential parameters and thus will not be sent. When `sndr` is not specified, `dsndr` is used.

** At least one (or more) of the `rcpt`, `rcpts` objects must be specified.

*** If the message does not contain `flgs`, the value of `dflgs` is used. If neither of them is defined, the value of 0 is used.

9.3 RestFul services – message sending

RestFul services *are* available at the following link:

<https://as.eurosms.com/api/v3/xxx/yyy>

The http *POST* method is used.

Xxx and yyy variants:

- **xxx** can take up the value of **send** or **test**:
 - **test** – does not actually send SMS. Instead, it is used for debugging clients' solutions and fully validates input formats (JSON),
 - **send** – this is a fully-valued sender, i.e., the validation takes place after which the individuals SMS are assigned to the queue.
- **yyy** – takes up the following values:
 - **one** – sending a single SMS to a number,
 - **o2m** – sending a single message to a group of numbers (see bulk SMS),
 - **m2m** – sending number of messages to multiple phone numbers by one request (see bulk SMS).

9.3.1 Request JSON object **one**

Call one is used for sending a single text message to a single phone number.

It is used for calls:

- <http://as.eurosms.com/api/v3/send/one>
- <http://as.eurosms.com/api/v3/test/one>

*Note: It is recommend to use **test** for the purpose of customer-driven development. Using this method leads to fully validated input that does lead to message sending.*

```
{
  "iid": "1-ABCDEF",
  "sgn": "dfsdf244323",
  "rcpt": 421903622237,
  "flgs": 3,
  "ttl": 600,
  "sندر": "Info",
  "sch": "2018-01-10 12:30",
  "txt": "Jedna sprava na jedno cislo"
}
```

The mandatory parameters are highlighted in **red**.

Failing to state any of the mandatory parameters will result in not accepting the message:

- **iid** – Integration ID assigned by EuroSMS,
- **sgn** – digital signature calculated from integration key, sندر parameters, rcpt, and txt,
- **rcpt** – – message recipient. It is sufficient to use abbreviated version of phone numbers (without the dialling prefix) for Czech and Slovak numbers,
- **sندر** – sender– text (in CZ text only) with maximum of 11 characters, or telephone number in its full format, i.e. including the international dialling prefix and excluding the initial 00, or including a + for messages that are being sent to other countries,
- **txt** – ttext of the message being sent to the recipient.

The optional parameters enable changes within the message processing behaviour:

- flgs – have an impact on the message length as well as the use of diacritical characters,
- ttl – time to live. Mostly used for the authentication messages. If the message recipient is not reachable by ttl, the message will be never delivered to him/her.

Warning: *This parameter is not supported by every operator.*

- sch – schedule time –schedules the earliest time to send messages. When there is a large number of messages scheduled to be sent at the same time, the messages will be sent in the order in which they were registered by the system.

It *is not* allowed to use the sch parameter for the individual **m2m** messages. The schedule time must be specified with one minute accuracy in the 2018-01-10 12:30 format.

Warning: *Operators do not support using more precise schedule time.*

9.3.2 Response JSON object **one**

The system responds with JSON object.

If the *JSON Request* is OK, the following response is returned:

```
{
  "uuid": [ "D0E17E57-F455-430E-844D-EAD6F88A18E4" ],
  "err_code": "ENQUEUED",
  "err_desc": "Message accepted and enqueued to send"
}
```

Uuid is an array of ID messages located in the EuroSMS systems. At later stage, these arrays contain information regarding the delivery status of the messages. It acts as a n array due to the technological segmentation (3.2 The fundamentals of Short Message Services and Segmentation).

Erroneous *JSON Request* can lead to the following:

- if the *JSON input* can no longer be parsed, the system error is returned,
- if the *JSON Request* is syntactically ok, its mandatory parameters and digital signature and so on and so forth are further tested,

For an instances where the input is incorrect, *Response JSON* **will not** contain uudi.

In order to facilitate easier error detection, the system attempts to return all of the detected errors in a single response.

```
{
  "err_code": "FAILED",
  "err_list": [
    {
      "err_code": "NO_BALANCE",
      "err_desc": "Account has low balance to send message."
    },
    {
      "err_code": "WRONG_SIGNATURE",
      "err_desc": "Signature does not match"
    }
  ]
}
```


9.3.3 Request JSON object **o2m**

A **o2m** call is used when a single text is sent to a multiple telephone numbers.

The maximum number of messages sent in a single transaction is **1000**.

It is used for calls:

- <http://as.eurosms.com/api/v3/send/o2m>
- <http://as.eurosms.com/api/v3/test/o2m>

Note: It is recommended to use test for the purpose of customer-driven development. Using this method leads to fully validated input, however, it does not result in message sending.

An example of JSON object of *o2m* type:

```
{
  "iid": "1-ABCDEF",
  "rsp": "basic",
  "sgn": "dfsdf244323",
  "rcpts": [ 421903622237, 902121212 ],
  "flgs": 3,
  "ttl": 600,
  "sndr": "Info",
  "start": "2018-01-10 12:30",
  "dur": "12:00",
  "txt": "Jedna sprava na viac cisel"
}
```

The differences between the *JSON request* and *ONE request* method are as follows:

- rcpt in the send/one method is replaced by the rcpts array,
- to calculate the digital signature the system takes into consideration all of the recipients' telephone numbers (this example only includes the two 421903622237 and 902121212) that are joined into a single string. The string does not contain any delimiters (e.g., "421903622237902121212") nor any text,
- must contain at least one of the recipients' phone numbers,
- can contain the *rsp* parameter which predefines how much detail will be included in the systems' response,
- does not contain *sch* parameter, or *sch* parameter be ignored
- can contain *start* (*scheduled time of the message sending*) and *dur* (*a parameter denoting the total of elapsed hours and minutes that passed since*

the process of message sending launched) parameters. Together these parameters define **when** the messages will be sent and **how long** it will take to deliver them.

9.3.4 Response JSON object o2m

Depending on the `rsp { basic | full }` values, JSON response can take up two formats:

- **basic** – will return only a statistical overview. Such overview contains information regarding the number of rejected and accepted messages by the system,
- **full** – will return complex information containing list of errors, identifiers assigned to individual phone numbers, etc.

9.3.4.1 BASIC JSON Response Format

```
{
  "accepted": 23,
  "rejected": 2,
  "group_id": 223123
}
```

9.3.5 FULL JSON Response – completely rejected

JSON request containing at least one error due to which it is impossible to carry on with the request, will be rejected. This happens when the `err_list` is non- empty. Additionally, request with an empty “accepted” list will be also fully rejected.

```
{
  "err_code": "FAILED",
  "err_list": [
    {
      "err_code": "NO_BALANCE",
      "err_desc": "Account has low balance to send message"
    },
    {
      "err_code": "WRONG_SIGNATURE",
      "err_desc": "Signature does not match"
    }
  ],
  "wrong_numbers": [
    {
      "r": 42190362
    },
    {
      "r": 429123423
    }
  ],
}
```

```
"accepted": []  
]  
}
```

9.3.6 FULL *JSON Response* – some of the telephone numbers are rejected

```
{  
  "err_code": "ENQUEUED",  
  "err_list": [],  
  "group_id": 223123,  
  "wrong_numbers": [  
    {  
      "r": 42190362  
    },  
    {  
      "r": 429123423  
    }  
  ],  
  "accepted": [  
    {  
      "r": 421903622237,  
      "i": [ "405B0C82-CF9A-4D95-80AB-41A75A6A5EE7" ]  
    }  
  ]  
}
```

9.3.7 FULL *JSON Response* – fully accepted

```
{  
  "err_code": "ENQUEUD",  
  "err_list": [],  
  "group_id": 2231232,  
  "accepted": [  
    {  
      "r": "421903622237",  
      "i": [ "3d7f0b97-3b26-4ace-a9c9-ee39bfc4cf07" ]  
    },  
    {  
      "r": "902121212",  
      "i": [ "405B0C82-CF9A-4D95-80AB-41A75A6A5EE7" ]  
    }  
  ],  
  "wrong_numbers": []  
}
```

9.3.8 Request JSON object **m2m**

The **m2m** call is used for sending variety of texts to number of different phone numbers. The maximum number of messages sent in one transaction is **1000**.

Essentially, it is a set of **one** messages sent after a single call to the Restful services. The Restful services do not require that you specify the mandatory information for each of the messages. Instead, pre-defining values of some attributes allows for using *default* versions of them.

It is used for calls:

- <http://as.eurosms.com/api/v3/send/m2m>
- <http://as.eurosms.com/api/v3/test/m2m>

Note: We recommend using **test** for the purpose of customer-driven development. Using this method leads to fully validated input, however, does not lead to message sending.

An example of **m2m** *JSON Request*:

```
{
  "iid": "1-ABCDEF",
  "dflgs": 2,
  "rsp": "basic",
  "dsndr": "Info",
  "start": "2018-01-10 12:30",
  "dur": "12:00",

  "msgs": [
    {
      "txt": "Jedna sprava",
      "flgs": 3,
      "sgn": "76656sfdgdg",
      "rcpt": 903622237,
      "sndr": "RobertZilka"
    },
    {
      "rcpt": 420771121212,
      "txt": "Jedna dlha sprava na ine cislo, Jedna dlha sprava
na ine cislo, Jedna dlha sprava na ine cislo, Jedna
dlha sprava na ine cislo, Jedna dlha sprava na ine
cislo, Jedna dlha sprava na ine cislo.",
      "sgn": "dfsdf244323"
    }
  ]
}
```

9.3.9 Response JSON object **m2m**

The **m2m** call is used for sending bulk texts to number of different phone numbers.

In bulk messages processing, Restful API estimates whether it is possible to send all the messages at once. However, since the final SMS price is calculated after the routing and billing transitions, there may be a situation where not all messages will be sent. In other words, exhausting clients' funds prior processing the entire batch will result in failure to send all the messages.

Depending on the `rsp { basic | full }` value, JSON response can take up the following formats:

- **basic** –will return statistical overview only. Such overview contains information regarding the number of rejected and accepted messages by the system,
- **full** – will return complex information containing list of errors, identifiers assigned to individual phone numbers, etc. Similarly to the **o2m**, list of accepted and rejected (`wron_numbers`) phone numbers is returned.

9.3.10 BASIC JSON Response format

```
{
  "accepted": 23,
  "rejected": 2,
  "group_id": 1232323
}
```

9.3.11 FULL JSON Response format

The system accepts JSON as a whole. The 'accepted' list must contain at least one phone number while the err_list is empty. To see other response examples, please see **o2m** JSON Response. A long (two segment) message was sent to the 420773773237 number. To see the segmentation details, please see -- 3.2 The fundamentals of Short Message Services and Segmentation.

```
{
  "err_code": "ENQUEUED",
  "err_list": [],
  "group_id": 3344123,
  "result": [
    {
      "e": "WRONG_NUMBER",
      "r": 42190362
    },
    {
      "e": "WRONG_SIGNATURE",
      "r": 429123423
    },
    {
      "e": "ENQUEUED",
      "r": 421903622237,
      "i": [ "405B0C82-CF9A-4D95-80AB-41A75A6A5EE7" ]
    },
    {
      "e": "ENQUEUED",
      "r": 420773773237,
      "i": [ "20CD7672-176C-47B8-8440-7EA3B11B2C57",
            "510b310a-ea3e-11e8-b780-87afcd091a1a"
          ]
    }
  ]
}
```

If any of the “i” keys contains “v:” prefix it acts as a message identifier for messages sent via Viber. Messages sent using the standard SMS route, contain alternative identifiers.

9.4 RestFul services – delivery status

9.4.1 Delivery status of a single message (operation **one**)

RestFul services are used to acquire delivery status of a single message by a single call. The RestFul services *are* available at:

<https://as.eurosms.com/api/v3/status/one/yyy>

The http *GET* method is used. The yyy message identifier for status/one cases can be obtained after the message in the UUID format is sent.

For example, 49bad057-38c7-4af1-85f7-94c39499545a.

This method can be called repeatedly, and it always returns the current status.

Message statuses for SMS API are stored in the production system for 7 days after their delivery. The storage time for reporting from the www.eurosms.com portal the messages are stored for 38 days.

Statuses of all segments (mostly 1 to 4 segments of originally long message) are returned for the multipart messages (long SMS).

If the Id does not exist, the returned *JSON Response* is in the object array format containing minimum of one object.

9.4.2 *status/one Response format*

```
{
  "rcpt": 421903170552,
  "carrier": "231.6",
  "price": 0.0265,
  "dlr_time": "2017-11-23 08:05:25",
  "sgmnt": 2,
  "dlr": "DELIVRD",
  "i": "be863034-df6b-5e3f-b681-649bf7017b9e",
  "snd": "2017-11-23 08:05:21",
  "err_code": "OK",
  "f": null
}
```

9.4.3 Bulk SMS delivery status (operation **any**)

RestFul services provide the delivery status of bulk messages, by a single call. The RestFul services are available at:

<https://as.eurosms.com/api/v3/status/any/xxx/trx/zzz>

The http *GET* method is used.

The meaning of the unknown variables:

- xxx – (*String*) is the integration ID (e.g., 2-ZZFFGG),
- trx – (*String*) is the transaction number – random number/String with the minimal length of 8 characters and maximal length of 64 characters,
- zzz – (*String*) is the digital signature of the transaction number (trx).

The operation returns the *JSON Response* containing the delivery status of all messages regardless of whether they were sent individually (one at the time) or as a bulk/group messages.

The *JSON Response* return format is identical to the operation GROUP(9.4.5 *JSON Response* format for status/group and status/any operations).

Advantages to the operation ONE:

- call ANY will return large number of delivery reports simultaneously,
- The delivery reports sent to the customers are processed by the system. Thus, calling ANY operation further will result in sending out no further delivery reports.

For the segmented-long SMS, the statuses of all segments/parts of the message are returned SMS (please see, 3.2 The fundamentals of Short Message Services and Segmentation) s). Please note, that the long messages are technologically split into 1 - 4 segments.

Although, the operation can be called repeatedly, only messages whose delivery status has changed, will be returned.

The delivery statuses of messages that reached their final stage will not be returned after redialling (DELIVRD, REJECTED, EXPIRED, ...). Thus, this operation serves to ascertain the status of bulk messages.

9.4.4 The status of group/bulk messages (operation **group**)

RestFul services are used to inquiry the delivery status of the group/bulk messages. These services are available at:

<https://as.eurosms.com/api/v3/status/group/xxx/yyy/zzz>

The http *GET* method is used.

The meaning of the unknown variables:

- xxx – (*String*) is the integration ID (e.g., 2-XXYYZZ),
- yyy – (*Long*) is the group_id, returned by the JSON Response operations o2m a **m2m**,
- zzz – (*String*) is the digital signature group_id.

The operation will return the *JSON Response* which contains the information regarding the SMS delivery time which is a part of bulk/group message.

The delivery statuses of all segments are returned for the segmented (long) messages (please see, 3.2The fundamentals of Short Message Services and Segmentation). Please note, that the long messages are technologically split into 1 - 4 segments.

Although, the operation can be called repeatedly, only messages whose delivery status has changed, will be returned.

The delivery statuses of messages that reached their final stage will not be returned after redialling (DELIVRD, REJECTED, EXPIRED, ...). Thus, this operation serves to ascertain the status of bulk messages.

9.4.5 JSON Response format for status/group and status/any operations

```
{
  "dlrs": [
    {
      "rcpt": 421902589458,
      "carrier": "231.2",
      "dlr_time": "2017-11-28 12:22:02",
      "f": null,
      "price": 0.032,
      "snd": "2017-11-28 12:22:03",
      "i": "e58e883f-4518-5ed7-b169-9e346f78503d",
      "sgmnt": 1,
      "dlr": "EXPIRED"
    },
    {
      "rcpt": 421915837327,
      "carrier": "231.6",
      "dlr_time": "2017-11-28 12:20:13",
      "f": null,
      "price": 0.037,
      "snd": "2017-11-28 12:20:13",
      "i": "3293a313-9490-575d-a8d1-0f8604409ae5",
      "sgmnt": 1,
      "dlr": "EXPIRED"
    }
  ],
  "count": 2,
  "err_code": "OK"
}
```

10 Instant messaging (IM)

10.1 Fundamentals

Calling *RestFul* services, SMS API sends messages to *Viber*.

Advantages of Viber in comparison to standard messages:

- sending long messages (up to 1000 characters) including diacritical characters,
- identical price for all operators within the same country,
- high penetration of Viber users (SK ~ 30%, HU ~ 40%, CZ > 20%),
- messages will be automatically send using the standard SMS route (e.g., shorter, or excluding diacritical characters) to the phone numbers with unavailable or never activated Viber,
- time To Live (TTL) setting. Messages undelivered to Viber to the present TTL are resent using the standard SMS route. The minimum of 15 seconds and maximum of 24 hours,
- each sender can have an assigned logo, under which the message will be visible to the recipient,
- it is possible to respond to messages, i.e., the recipient can respond to the delivered message,
- unlike SMS, there is no charge for undelivered Viber messages,
- message recipients can unsubscribe from receiving messages. This will result in immediate action that leads to Viber disallowing the senders to send message to specific phone numbers. Thus, the message may (may not) be delivered as standard SMS.

10.2 IM sending conditions

Senders need to meet the above- standard settings and conditions to be able to send messages via Viber (not indicated for SMS in its regular format sent to most countries):

- free sender registration used in Slovakia.
(Countries such as CZ, PL, RO etc., have their SMS sender registration charged.),
- attaching the senders' picture/logo (recommended but not required),
- sender registration can take up to two weeks and must be accepted by Viber. At the time of registration, it is required to make decision regarding reception of message replies.
Changes during the operation are subject to an approval process (which can take up to 2 weeks),
- the cost of messages *delivered* by Viber **must** range between 150 and 300€ (this is country dependent).
The pricing for messages is cumulative for messages sent to multiple countries by a single sender. For example, in Slovakia it is recommended to send approximately 100 thousand SMS monthly. Senders who do not meet the criteria set by Viber regarding the minimum of delivered messages are charged at a flat minimum rate,
- the customer/recipient can *unsubscribe* from the sender. As a result, the Viber messages will be undeliverable and will be sent using the standard SMS route,
- The EuroSMS systems are allowed to send a message by Viber only after Viber successfully completes the necessary administrative actions.

Thus, attempting to send a message by Viber prior completing those processes will result in rejecting to send the message, or sending the message using the standard SMS route.

Warning: *JSON Request* that contains an IM section and a sender who is not registered or is registered for another user is identified, such sender will be replaced by already registered version.

10.2.1 An example of JSON Request for an IM sending

Warning: The following is an example of extended IM which contains variants of the JSON values. *It is important to note, that this example is illustrative only and cannot be used to send a message.* The individual Restful services variants such as – **one**, **o2m**, **m2m**, are described earlier in this document.

An abstract example of *JSON Request* object.
The structures highlighted in blue are an *IM* specific.

```
{
  "iid": "1-ABCDEF",
  "dflgs": 2,
  "rsp": "basic",
  "dsndr": "Info",
  "start": "2018-01-10 12:30",
  "end": "2018-01-10 16:30",
  "dur": "12:00",
  "dmsndr": "Registrovany Sender",
  "dimttl": 1800,
  "msg": {
    "rcpt": 421903622237,
    "txt": "Alternativna sprava na jedno cislo",
    "flgs": 3,
    "sgn": "dfsdf244323",
    "sndr": "RoboZilka",
    "im": {
      "sndr": "Registered Sender",
      "ttl": 600,
      "msg": "Úplný text posielaný s diakritikou, max. 1000 znakov."
    }
  },
  "msgs": [
    {
      "txt": "Jedna sprava viacerym odosielateľom.",
      "flgs": 3,
      "sgn": "76656sfdgdg",
      "rcpts": [
        0903622237,
        420766121212,
      ],
      "sndr": "RoboZilka",
      "im": {
        "sndr": "Registered Sender",
        "ttl": 2400,
        "msg": "Úplný text skupiny posielaný s diakritikou, max. 1000 znakov."
      }
    },
    {
      "rcpt": {
        "r": 4412345678,
        "e": "21C334C7-CD7E-45D7-AA88-FC9C58A7FF47"
      },
      "txt": "Jedna sprava na ine cislo",
      "sgn": "dfsdf244323",
      "im": {
        "sndr": "Registered Sender",
        "ttl": 1800,
        "msg": "Úplný hromadný text posielaný s diakritikou, max. 1000 znakov."
      }
    }
  ]
}
```

The table shown above contains Viber arrays. The *im* object is not mandatory. However, an introduced *im* object *must* be non-empty.

10.3 JSON object of **im** type

Abbreviation	Title	Type	Description	Mandatory
sndr	Registered sender	String	Sender, as registered in Viber. It is not „case sensitive“. It is automatically revised and corrected into the registered version, if necessary.	✗
ttl	Time To Live	Integer	This value is expressed in seconds. It refers to the overall time that the system is required to wait prior sending an alternative text using the standard SMS route. If not specified, its value is 3.600 (1 hour). Minimum of 15 seconds. Maximum of 24 hours.	✗
msg	Text message sent by Viber	String	Text message sent by Viber can contain maximum of 1000 characters including diacritics	✓

10.4 Default IM values – an extension of the **root** object

In order to avoid listing all information in the *im* object, it is possible to define some information as *default* in the root of the structure.

Abbreviation	Title	Type	Description	Mandatory
dimsndr	Registered sender	String	Sender, as registered in Viber. If not specified within the <i>im</i> ttl object, the <i>dimttl</i> is used. If neither is defined, the system does not send the message. Instead, the message is marked as incorrect. It is not „case sensitive“. It is automatically revised and corrected into the registered version, if necessary.	✗
dimttl	Time To Live	Integer	This value is expressed in seconds. It refers to the overall time that the system is required to wait prior sending an alternative text using the standard SMS route. If not specified within the <i>im</i> ttl object, the <i>dimttl</i> is used. If neither is defined, the system uses the default value of 3600. If not specified, its value is 3.600 (1 hour). Minimum of 15 seconds. Maximum of 24 hours.	✗

10.5 Flags specific to IM

Behaviour of the messages sent to Viber is significantly influenced by *flags*, which can take on an additional value (ignored by standard sms).

Flag Name	Value	Description
FLAG_IM_PROMO	256	Viber allows for Promo/Marketing message. However, such messages must be marked as PROMO. <i>Warning 1:</i> Viber has the right to block even registered senders who send the Promo/Marketing messages unmarked <i>Warning 2:</i> Using Viber on smartphones, each recipient has the right to block reception of messages of any sender.
FLAG_IM_ONLY	128	Messaged undelivered to Viber are marked with this flag. The system does not further process this request; therefore the message will not be sent using the standard SMS route. Recommended to use with the FLAG_IM_PROMO flag.

10.6 Extension of error codes by specific IM statuses for SMS sending

The extension of 13.1 The list of SMS and IM send/delivery statuses .

Error Code	Error Description
IM_NOT_ALLOWED	Sending of IM messages for specific account is not allowed
IM_UNREGISTERED_SENDER	The sender is incorrect – no registered sender was found
IM_MSG_EMPTY	An empty IM message is not sent.
IM_MSG_TOO_LONG	IM message is too long > 1000 characters.
IM_TTL_OUT_OF_RANGE	TTL is out of the 15 seconds to 24 hours range.

10.7 JSON Response – – the system response to the IM Request

JSON Response to the IM Request has an identical format to the messages sent using the standard SMS route.

The only difference is in the “v:” prefix, which is preliminary to the Viber message identifier.

10.8 IM delivery statuses – the extension of delivery statuses

Delivery statuses and methods used to obtain the information regarding the delivery status are identical to the standard SMS. Viber also provides the information regarding whether the recipients read the message.

The list is extended by a single status.

Status	Description	Final
SEEN	The IM message recipient (most likely) read the message on their smartphone.	✓

10.9 Messages received by Viber

Recipients receiving messages on Viber can respond to the senders. However, recipients can only reply to the senders if they specified this in the registration request.

Received messages will be delivered into the customer services in their standard formats such as SMS, i.e., CallBack (Push), mail, or SMPP. While messages delivered using the CallBack and mail method will be delivered as a whole, messages exceeding the standardised message length delivered using the SMPP method will be segmented (160 ascii characters, 70UFT).

11 Appendix A

11.1 Examples of digital signature calculation using the SHA1 (Java) algorithm

SHA1 calculation

```
/**
 * Vypocita signaturu v3, algoritmus HmacSHA1.
 *
 * @param integration_key - moze byt c_id, u_id, u_integration_id, c_integration_id,
 *                          podla toho, co zakaznik poslal
 * @param toSign - vypočet: src + dst(s) + message
 * @return
 */

public static String createEurosmsV3Signature(String integration_key, String toSign) {
    String ret;
    try {
        SecretKeySpec signingKey =
            new SecretKeySpec(integration_key.getBytes(), HMAC_SHA1_ALGORITHM);
        Mac mac = Mac.getInstance(HMAC_SHA1_ALGORITHM);
        mac.init( signingKey );
        ret = toHexString(mac.doFinal(toSign.getBytes()));
    } catch (Exception e) {
        ret = null;
    }
    return ret;
}

private static String toHexString(byte[] bytes) {
    Formatter formatter = new Formatter();
    for (byte b : bytes) {
        formatter.format("%02x", b);
    }
    return formatter.toString();
}
```

SHA1 algorithm using simple calculation

```
public static String createEurosmsV3Signature(String integration_key, String toSign)
{
    try {
        SecretKeySpec signingKey =
            new SecretKeySpec(integration_key.getBytes(), HMAC_SHA1_ALGORITHM);
        Mac mac = Mac.getInstance(HMAC_SHA1_ALGORITHM);
        mac.init( signingKey );
        String sign = (new BigInteger(1, mac.doFinal(toSign.getBytes()))).toString(16);
        return "0000000000000000".substring(0, 40 - ret.length()) + sign;
    } catch (Exception e) {
        return null;
    }
}
```

12 Appendix B

The GSM 3.38 character set is optimised for SMS sending and it is set as a default by most of the operators and aggregators.

It is important to note that **IM** strictly uses the UTF-8 character set.

All characters including special characters originating from Cyrillic, Hebrew, and Russian alphabet etc., are deliverable.

Each character set is represented by 7-bits and can be extended by other symbols. The characters originating from the extended set are represented by 14-bits, where the initial 7 bits mark the extension (ESC = 0x1B).

12.1 Basic character set GSM 3.38

	0x00	0x10	0x20	0x30	0x40	0x50	0x60	0x70
0x00	@	Δ	SP	0	i	P	ı	p
0x01	£	—	!	1	A	Q	a	q
0x02	\$	Φ	"	2	B	R	b	r
0x03	¥	Γ	#	3	C	S	c	s
0x04	è	Λ	¤	4	D	T	d	t
0x05	é	Ω	%	5	E	U	e	u
0x06	ù	Π	&	6	F	V	f	v
0x07	ì	Ψ	'	7	G	W	g	w
0x08	ò	Σ	(	8	H	X	h	x
0x09	Ç	Θ	)	9	I	Y	i	y
0x0A	LF	Ξ	*	:	J	Z	j	z
0x0B	Ø	ESC	+	;	K	Ä	k	ä
0x0C	ø	Æ	,	<	L	Ö	l	ö
0x0D	CR	æ	-	=	M	Ñ	m	ñ
0x0E	Å	ß	.	>	N	Ü	n	ü
0x0F	å	É	/	?	O	§	o	à

12.2 The extension of the GSM 3.38 set

In contrast to the fixed ASCII set, extending character allows for sending special characters such as €.

Warning: including single characters such as € will lead to extending the SMS length. If the message length is n and it is visible, the physical length of the message corresponds to $n+1$. Messages containing 160 characters and an additional single character € will be 161 characters in length. Therefore, such message will be split into 2 segments and charged as 2 billing units (for more details, please see 3.2 The fundamentals of Short Message Services and Segmentation).

	0x00	0x10	0x20	0x30	0x40	0x50	0x60	0x70
0x00								
0x01								
0x02								
0x03								
0x04		^						
0x05								
0x06						€		
0x07								
0x08			{					
0x09			}					
0x0A	FF							
0x0B		SS2						
0x0C				[				
0x0D	CR2			~				
0x0E				]				
0x0F								

13 Appendix C

13.1 The list of SMS and IM send/delivery statuses

Error Code	Error Description
ENQUEUED	Message accepted and enqueued to send. Note.: <i>A single positive response.</i> If the <i>JSON Request</i> is ok, the ENQUEUED response suggests that the request will be further processed and the SMS will be sent.,
NO_IID	JSON doesn't contain key integration id
NO_MSG	JSON doesn't contain key message to send
NO_RCPT	JSON doesn't contain key for recipients
NO_TXT	JSON doesn't contain key for text
NO_SGN	JSON doesn't contain key for signature
NO_SNDR	JSON doesn't contain key for sender
NO_BALANCE	Account has low balance to send message
WRONG_SIGNATURE	Signature does not match
WRONG_IID	Integration id is wrong or unknown
WRONG_NUMBER	Wrong format of phone number
WRONG_SENDER	Sender is not correct (too long, too short, etc.)
EMPTY_MESSAGE	Message doesnt contain any characters
TOO_MANY_MESSAGES	Attempting to send too many messages Note: since the incoming JSON is reviewed online, the number of sms sent by a single <i>JSON Request</i> will be limited to a thousand .
MSG_TOO_LONG	Message have too many characters Note: <i>JSON Request</i> will assign this error code to the messages that are longer than 160 characters or their <code>flgs</code> do not have the value of <code>0x02</code> .
ERR_OTHER	Other error Note: Any other potential errors will be marks as <code>ERR_OTHER</code> .

13.2 Full list of SMS, IM delivery states

Status	Description	Final
ENROUTE	The messages entered the operators' queue.	✗
ACCEPTD	Message accepted by the operators' SMS Centre and its awaiting to be sent.	✗
DELIVRD	Delivered..	✓
UNDELIV	Undelivered (non-existing telephone number, etc.)	✓
EXPIRED	Undelivered within the maximum of 7 days. The expiration date varies for each operator.	✓
REJECTD	Rejected by operator, (e.g., the customer failed to pay, the phone number has been turned on for prolonged period of time, etc.)	✓
DELETED	SMS cancelled by the operators' Centre.	✓
UNKNOWN	Operators' unknown error.	✓
SEEN	The recipient saw the IM message and most likely read it on their smartphone. IM only.	✓

14 Appendix D

14.1 Flags – characteristics impacting SMS API behaviour

Each message sent by any method is influenceable by flags.

Flags data type is Integer.

The final value will be calculated as the sum of individual values, where each value is an expression of a bit.

Title	Hex	Dec	Description / meaning
Delivery report	0x001	1	Request to generate delivery report by the operator.
Long message	0x002	2	Expected message will be longer than 160 or 70 characters, respectively. Only the actual length of the message will be charged. Attempting to send a long message without setting up this flag will result API rejection to send the message.
Diacritics	0x004	4	PUtf-8 permission for the use of diacritical characters. Setting up this flag will result in technological shortening of the message to the 70 characters or 67 characters for a long message. Special characters such as ščťžýáíäô, Cyrillic, Arabic script, and/or Chinese characters will be accepted. Not setting up this flag and using special characters will result in replacing such characters by the "??".
Higher priority	0x008	8	Increased priority for the message sent. <i>Warning:</i> messages with higher priority are charged at higher rate.
Lower priority	0x020	32	Mostly used for the campaign messages such as bulk marketing messages with identical text.
Viber	0x200	512	The message is initially sent to Viber. Messages sent to the phone numbers that do not have activated Viber will be after predefined time (this can be set up individually or as a default for all of the customers' messages) sent as a standard SMS.

			The maximum length of the message with diacritical characters sent on Viber is 1000 characters. In contrast to standard SMS, such message will be charged as a single segment.
Viber only	0x080	128	Mostly used in the combination with Viber flags. When set, messages that were undelivered on Viber will not be further forwarded using the standard route.
Viber promo	0x100	256	It is mandatory for Promo messages sent on Viber. At the same time, the customers must obtain permission (e.g., after signing a loyalty agreement, card, or club memberships) to send marketing messages to the specific number. <i>Viber has the right to fine the authors of the SPAM messages after beneficiaries file complaints against SPAM messages on Viber. The authors will be fined 5 thousand EUR for each of the complaint.</i>

14.2 Flags – calculation examples

The required form of Flags for specific case is calculated as a sum of individual flags:

- standard short message with delivery report:
 $Flags = 1$
- long message with diacritical characters (length of e.g., 200 characters):
 $Flags = 1 + 2 + 4 = 7$
- long campaign message, excluding diacritical characters:
 $Flags = 1 + 2 + 32 = 35$
- long message excluding diacritical characters, using primary Viber, with a SMS backup:
 $Flags = 1 + 2 + 512 = 515$